Modern Operating Systems

Modules 3 & 4

Deadlocks · Memory · Files · Storage

Understand the mechanism. Practise the trace. Explain the trade-off.

Expanded self-study edition, with introductory examples, step-by-step traces and topic-specific GeeksforGeeks links.
This guide develops every topic listed for Modules 3 and 4 in your department curriculum. It combines connected explanations, diagrams you can redraw, complete worked examples, answer structures, and a practice set with solutions.

Module 3	Module 4	Examination
10 contact hours	8 contact hours	60 marks; 2 h 30 min
Deadlock handling	Memory, files, storage	15 marks per module

Reading and print edition • Prepared 7 October 2026

Contents

Scope, syllabus and how to study 1

I Module 3: Deadlocks 3

1 What a deadlock means 3
1.1 A two-lock example . . . 3
1.2 Deadlock, starvation and livelock . . . 3

2 The four necessary conditions 4
2.1 Test the conditions against an actual snapshot . . . 4
2.2 A deadlock-free resource-count bound: supporting numerical skill . . . 4

3 Resource-allocation graphs 5
3.1 One instance per type: a cycle proves deadlock . . . 5
3.2 Multiple instances: a cycle is not enough . . . 6

4 Deadlock prevention 6
4.1 Remove mutual exclusion where possible . . . 6
4.2 Remove hold and wait . . . 6
4.3 Allow resource preemption . . . 6
4.4 Remove circular wait by ordering . . . 7

5 Avoidance and safe states 7
5.1 Single-instance avoidance using claim edges . . . 7
5.2 Learn safe and unsafe with one resource type . . . 8
5.3 Why claim edges differ from current requests . . . 8
5.4 Read the safety algorithm as a proof search . . . 8

6 Banker's algorithms 8
6.1 Data and assumptions . . . 9
6.2 The safety algorithm . . . 9
6.3 A complete safety example . . . 9
6.4 The resource-request algorithm . . . 10

7 Deadlock detection 11
7.1 One instance: a wait-for graph . . . 11
7.2 Multiple instances: the matrix algorithm . . . 11
7.3 When should detection run? . . . 12

8 Recovery from deadlock 12
8.1 Process termination . . . 12
8.2 Resource preemption and rollback . . . 12
8.3 Detection and recovery form a complete strategy . . . 12

9 Choosing a deadlock strategy 13

II Module 4A: Memory management 14

10 Address spaces and address binding **14**
10.1 When binding occurs . . . 14
10.2 Base and limit: the simplest translation . . . 14
10.3 Follow an address from source code to execution . . . 15

11 Swapping **15**

12 Contiguous allocation and partitions **16**
12.1 Fixed partitions . . . 16
12.2 Variable partitions . . . 16
12.3 First fit, best fit and worst fit . . . 16
12.4 Coalescing and compaction . . . 17
12.5 Count allocated slack and free partitions separately . . . 17
12.6 See what compaction moves . . . 18

13 Segmentation **18**
13.1 Segment-level protection and sharing . . . 19

14 Paging, page tables and the TLB **19**
14.1 Translating an address . . . 19
14.2 Address bits, table size and fragmentation . . . 20
14.3 What a PTE contains . . . 20
14.4 The translation lookaside buffer . . . 20
14.5 Page-table organisation: supporting ideas . . . 21
14.6 Why multilevel page tables save space . . . 21
14.7 Hashed and inverted tables: understand the lookup key . . . 22
14.8 Paging versus segmentation . . . 22

15 Virtual memory and demand paging **22**
15.1 Servicing a page fault . . . 22
15.2 Paging and demand paging answer different questions . . . 23
15.3 Build a reference string from byte addresses . . . 23
15.4 Why a dirty victim can make a fault slower . . . 23
15.5 Fault cost and effective access time . . . 23
15.6 Locality, working sets and thrashing . . . 24
15.7 Frame allocation and the working-set argument . . . 24
15.8 Copy-on-write: a useful supporting example . . . 25

16 Page-replacement algorithms **25**
16.1 FIFO: replace the oldest load . . . 25
16.2 Optimal: replace the farthest future use . . . 25
16.3 LRU: replace the longest unused page . . . 25
16.4 Placement, replacement and allocation are different decisions . . . 25
16.5 Explain the trade-off behind the replacement rule . . . 25
16.6 One verified reference string, three complete traces . . . 26
16.7 Second chance and Clock . . . 28
16.8 Other replacement ideas at revision depth . . . 29
16.9 Belady's anomaly . . . 29

III Module 4B: File systems **31**

17 The file concept **31**
17.1 Attributes: what the OS records . . . 31
17.2 Operations: what an application can request . . . 31
17.3 File types and internal structure . . . 32
17.4 Separate a file's identity, type and contents . . . 32
17.5 A file-operation trace . . . 33

18 File access methods **33**
18.1 Sequential access . . . 33
18.2 Direct or random access . . . 33
18.3 Indexed access . . . 33
18.4 Indexed sequential access combines lookup and traversal . . . 34

19 File protection **34**
19.1 Permission bits and access-control lists . . . 34
19.2 Protection, sharing and integrity . . . 34
19.3 Visualise rights with an access matrix . . . 35

20 File-system implementation **35**
20.1 A layered view . . . 35
20.2 On-disk and in-memory structures . . . 36
20.3 Following open, read and close . . . 36
20.4 Walk through a read as an implementation problem . . . 36
20.5 Descriptors, open objects and file identity . . . 37
20.6 Mounting connects a volume to a namespace . . . 37
20.7 Free-space management . . . 37
20.8 Practise a bitmap and a counted free-run representation . . . 38

21 Directories and their implementation **38**
21.1 Logical directory organisations . . . 38
21.2 Implementing entries: linear list versus hash table . . . 39
21.3 Hard links and symbolic links: supporting distinctions . . . 39
21.4 Evaluate directory designs by naming, grouping and sharing . . . 39
21.5 Directory organisation and lookup implementation are different . . . 39

22 File allocation methods **40**
22.1 Contiguous allocation . . . 40
22.2 Linked allocation . . . 40
22.3 Indexed allocation . . . 40
22.4 See the indexed mapping explicitly . . . 41
22.5 Pointer overhead can change the required block count . . . 41
22.6 Locate a block through direct and indirect pointers . . . 41
22.7 Comparison you can use in an exam . . . 42

IV Module 4C: Storage management **43**

23 Magnetic disks and disk structure **43**
23.1 The geometry vocabulary . . . 43
23.2 Where an access spends time . . . 43
23.3 Distinguish capacity, bandwidth and latency . . . 44

24 Solid-state disks **44**
24.1 Flash pages, erase blocks and the FTL . . . 44
24.2 Follow an overwrite through the flash mapping . . . 45

25 Disk scheduling **46**
25.1 The six standard algorithms . . . 46
25.2 A complete checked scheduling problem . . . 46
25.3 A redrawable head-path diagram . . . 47
25.4 Comparing algorithms fairly . . . 47
25.5 Reconstruct each service policy from a decision rule . . . 48
25.6 Separate finite-snapshot answers from ongoing behaviour . . . 48

26 Formatting, partitions and boot information **48**
26.1 Low-level or physical formatting . . . 48
26.2 Partitioning . . . 48
26.3 Logical formatting . . . 49
26.4 Booting and bad-block handling . . . 49
26.5 Follow one drive through the preparation stages . . . 49

V Revision and exam practice **50**

27 Module 3: rapid revision **50**
27.1 Three answer structures . . . 50

28 Module 4: rapid revision **50**
28.1 Memory . . . 50
28.2 Files and storage . . . 51

29 Formula sheet **52**

30 Common mistakes and the correct statement **53**

31 Part A practice: short answers with solutions **55**

32 Part B practice: longer answers and numericals **58**
32.1 Problem 1: prevention and its proof . . . 58
32.2 Problem 2: a fresh Banker snapshot . . . 58
32.3 Problem 3: multiple-instance detection . . . 58
32.4 Problem 4: paging and table size . . . 58
32.5 Problem 5: TLB and fault performance . . . 59
32.6 Problem 6: page replacement by hand . . . 59
32.7 Problem 7: file allocation and metadata . . . 59
32.8 Problem 8: a fresh disk scheduling queue . . . 59
32.9 Problem 9: device mechanisms and formatting . . . 60

33 A final self-check **60**

34 Sources and the best resources for further study **61**
34.1 Use resources in this order . . . 61
34.2 Prescribed textbook: chapter map and official materials . . . 61
34.3 Tanenbaum and Bos: an excellent reference . . . 61

34.4 Free textbook: OSTEP, by topic 62
34.5 University notes: a compact alternate explanation 62
34.6 Supplementary practice and video resources 63
34.7 Preparation and verification record 63

35 GeeksforGeeks topic map and expanded lessons 64
35.1 Module 3: topic links 64
35.2 Module 4: topic links 64

Scope, syllabus and how to study

The controlling syllabus is *S3–S8 Electrical and Computer Engineering Final Syllabus*, already present in your CET folder, PDF pages 171–174. The course title is **Modern Operating Systems**, code **PEEOT521**. The syllabus extract is supplied alongside this guide. Older MOS PDFs were inspected as background; their claims were not treated as evidence.

Module 3 lists necessary conditions, resource-allocation graphs, deadlock prevention, avoidance using Banker's algorithms, detection, and recovery. **Module 4** lists address spaces, swapping, contiguous allocation, fixed and variable partitions, segmentation, paging, virtual memory, demand paging, page replacement, file concepts and protection, file-system and directory implementation, allocation methods, magnetic disks, SSDs, disk structure, disk scheduling, and formatting.

Examination pattern verified from the syllabus

Part A: two compulsory 3-mark questions from each module. Part B: choose one complete 9-mark question from two for each module, with at most three subdivisions. Thus each module contributes $2 \times 3 + 9 = 15$ marks; Modules 3 and 4 together contribute 30 of 60. The syllabus does not publish a topic-wise mark distribution. Practice questions in this guide are original practice, not claimed past-paper questions or predictions.

A practical study sequence

1. Read the definitions and the diagrams first. Explain each mechanism aloud without looking at the page.
2. Work through Banker, detection, allocation, paging, replacement and disk scheduling with the answers covered. The tables show enough intermediate steps to locate a mistake.
3. Prepare a 3-mark answer using a definition and two relevant points. Prepare a 9-mark answer using the mechanism, a labelled diagram or algorithm, an example and limitations.
4. Finish with the revision sheets and the practice set. Use the topic map below to check coverage.

Binary units are explicit: $1\,\text{KiB} = 1024$ bytes and $1\,\text{MiB} = 2^{20}$ bytes. All resource-vector comparisons are component-wise. Numerical algorithms assume the stated fixed request/reference sequence; different assumptions may give different traces.

Complete syllabus coverage map

Syllabus topic	Where to study
Necessary deadlock conditions	Section 2
Resource-allocation graphs	Section 3
Deadlock prevention	Section 4
Avoidance; Banker's algorithms	Sections 5–6
Detection; recovery	Sections 7–8
Concept of address spaces	Section 10
Swapping	Section 11
Contiguous allocation; fixed and variable partitions	Section 12
Segmentation; paging	Sections 13–14
Virtual memory; demand paging	Section 15
Page-replacement algorithms	Section 16
File concept: attributes, operations, types, structure	Section 17
Access methods; protection	Sections 18–19
File-system implementation	Section 20
Directory implementation	Section 21
File allocation methods	Section 22
Magnetic disks; disk structure	Section 23
Solid-state disks	Section 24
Disk scheduling; formatting	Sections 25–26

Supporting material. TLBs and page tables explain paging; free-space management explains file-system implementation; locality and thrashing explain virtual-memory behaviour. These are included at introductory depth. RAID, full access-matrix theory, network security and detailed OS case studies are not separately listed in these two modules.

Resources. The prescribed book is *Operating System Concepts*, 9th edition, Silberschatz, Galvin and Gagne (Wiley India, 2015). Chapter names and verified links appear in Section 34. Text and examples here are independently written; the conventional Belady reference string is identified where used. Numerical results are recomputed by the accompanying Python script.

Expanded GfG review. Relevant article sections and examples were compared with each topic group. Additional lessons explain the intermediate reasoning, not just the final definitions. Section 35 links each topic to its article; the source bundle records the changes and their checks.

I Module 3: Deadlocks

1 What a deadlock means

An operating system allocates finite resources to processes. A resource may be a printer, a memory buffer, a database lock or a mutex. A *resource type* is a category; an *instance* is one interchangeable unit of that category. Two equivalent printers are two instances of one type. A colour printer and a printer with different capabilities may need separate types.

Definition

A set of processes is deadlocked when every process in the set is waiting for an event that can be caused only by another process in that same set. In the reusable-resource model, each waits for a resource whose release depends on progress by another member of the set.

The normal resource lifecycle is **request** → **use** → **release**. If the requested instance is unavailable, the process waits. Deadlock arises when the dependencies close into a situation from which none of the participating processes can proceed unaided. Other processes outside the set may still execute.

1.1 A two-lock example

Thread T_0 takes lock A, then attempts lock B. Thread T_1 takes B, then attempts A. If both first acquisitions happen before either second acquisition, each thread keeps a lock the other needs.

Time	T_0	T_1
1	Acquires A	—
2	—	Acquires B
3	Requests B; waits	Holds B
4	Holds A	Requests A; waits

The bug depends on execution order. The same program may appear to work whenever one thread gets both locks before the other starts. A single thread can also deadlock on a non-recursive lock it already holds.

1.2 Deadlock, starvation and livelock

Failure	What prevents progress?	Example and remedy
Deadlock	A closed set of waits cannot resolve itself.	Opposite lock order; impose consistent ordering or recover.
Starvation	A process is repeatedly bypassed while others proceed.	Unfair lock admission; use fair queuing or aging.
Livelock	Processes keep reacting, but their reactions prevent useful work.	Two contenders repeatedly release and retry together; use asymmetric rules or backoff.

Ordinary blocking is not automatically deadlock: a process may simply be waiting for a running process to finish an I/O operation. Deadlocked threads may sleep or spin, so "they never consume CPU" is not part of the definition.

Further reading: OSC 9e, Chapter 7, System Model; OSTEP Chapter 32, Concurrency Bugs.

2 The four necessary conditions

For deadlock involving reusable resources, the following conditions must hold together.

1. **Mutual exclusion.** At least one resource cannot be shared simultaneously. A mutex or an active printer is used by one owner at a time.
2. **Hold and wait.** A process holds at least one resource while waiting for an additional resource held by another process.
3. **No preemption.** A held resource cannot be forcibly removed; its owner releases it voluntarily. This refers to resource ownership, not CPU time-slice preemption.
4. **Circular wait.** There is a chain $P_0, P_1, \ldots, P_k$ in which each process waits for a resource held by the next, and P_k waits for one held by P_0.

In the two-lock example, the locks are exclusive, both threads hold one while requesting another, neither lock can be taken away safely, and the dependencies are circular.

Necessary is a precise word

Removing any one condition is sufficient to prevent this class of deadlock. Merely knowing that a system *allows* all four conditions does not prove it is currently deadlocked. With several instances per resource type, even a cycle in the resource-allocation graph may resolve through an instance released by another process.

Useful memory cue: exclusive resources, hold while waiting, cannot take away, waits form a circle. In an exam, write the actual condition names and explain them rather than giving only a mnemonic.

2.1 Test the conditions against an actual snapshot

Do not identify deadlock by the word "waiting" alone. A print request waiting behind a printer owner can finish when that owner finishes; there need be no dependency back to the waiter. For a cycle, follow *who must act to supply each missing event or resource.*

In the two-lock snapshot, T_0 cannot finish its protected operation without B, which T_1 owns. T_1 cannot finish without A, which T_0 owns. Neither can reach release. The important point is not simply that two locks exist, but that ownership and the acquisition sequence have closed the dependency chain.

CPU preemption does not fix this ownership cycle. Giving each task a new CPU turn only lets it discover that the required lock is still held. Resource preemption would need to remove a lock safely and repair any inconsistent state; that is a different operation.

2.2 A deadlock-free resource-count bound: supporting numerical skill

A common hand problem has n processes and *one type* of interchangeable resource, with each process needing at most k units simultaneously before finishing and returning all units. Under

this model, the minimum total that guarantees deadlock cannot occur solely from these claims is

$$R_{\min} = n(k-1) + 1.$$

Derivation. If nobody yet has enough to finish, each can hold at most $k-1$, so together they hold at most $n(k-1)$. One additional unit lets at least one unfinished claimant advance to its maximum if all are at that worst distribution; more generally, sufficient free units exist for at least one remaining claim. Once that process finishes, its releases permit further completions. With exactly $n(k-1)$ units, each can hold $k-1$ and wait for one more, creating a possible deadlock.

For unequal positive maxima M_i, the corresponding worst-case bound is

$$R_{\min} = 1 + \sum_i (M_i - 1).$$

If every unfinished process still lacked at least one of its needed units, the number free would be less than each remaining need. Combining that shortage with current allocations would force the total to be at most $\sum_i (M_i - 1)$; the larger total contradicts the supposed shortage for everyone.

Worked example: Four processes, each requiring at most three identical units

The guarantee requires $4(3-1)+1 = 9$ units. With 8, the distribution $(2, 2, 2, 2)$ leaves none free; all four can request one more and block. With 9, this distribution leaves one free, enabling a completion and release. Fewer than 9 units can still be safe in particular states; 9 is the worst-case guarantee under the assumptions.

Do not apply this formula by summing unrelated resource types. It assumes one interchangeable type, finite completion/release once a claim is met and no other dependencies. Banker's algorithm evaluates the actual multi-type snapshot rather than replacing it with this bound.

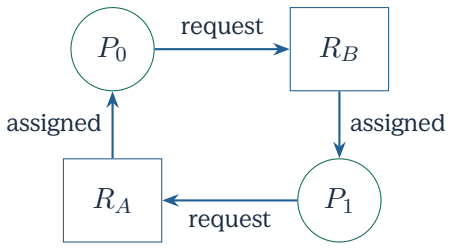
3 Resource-allocation graphs

A resource-allocation graph (RAG) is a directed graph whose vertices represent processes and resource types. Draw processes as circles, resource types as rectangles, and instances as dots within a rectangle.

- **Request edge $P_i \to R_j$:** P_i has requested an instance of R_j and is waiting for it.
- **Assignment edge $R_j \to P_i$:** a particular instance of R_j is held by P_i. The edge originates at its instance dot.

Granting a request replaces its request edge with an assignment edge in the opposite direction. Releasing the resource removes the assignment edge.

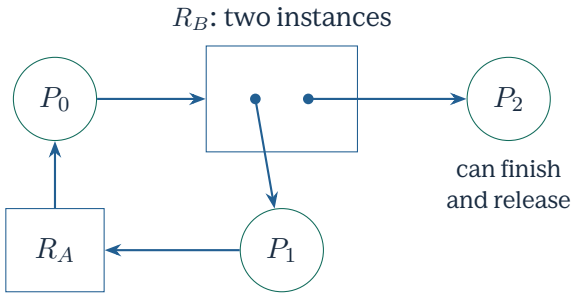
3.1 One instance per type: a cycle proves deadlock



Each rectangle denotes one instance here. R_A is held by P_0; R_B by P_1. The cycle $P_0 \to R_B \to P_1 \to R_A \to P_0$ cannot be broken by normal execution.

3.2 Multiple instances: a cycle is not enough

Suppose R_A has one instance and R_B has two. P_0 holds R_A and waits for R_B; P_1 holds one R_B and waits for R_A; P_2 holds the other R_B and needs no additional resource. The graph contains the same P_0–P_1 cycle. Nevertheless P_2 can finish and release its R_B; P_0 then finishes and releases R_A, allowing P_1 to finish.



Cycle rules to reproduce exactly

No directed cycle: no deadlock in this model.
Cycle, one instance of every resource type: deadlock.
Cycle, multiple instances allowed: deadlock is possible; analyse availability and completion rather than declaring it solely from the cycle.

An undirected loop is not enough. Follow arrow directions and identify which process can release each awaited instance.

4 Deadlock prevention

Prevention restricts how processes obtain resources so that at least one necessary condition cannot hold. It makes a structural rule, rather than checking the future safety of every tentative allocation.

4.1 Remove mutual exclusion where possible

Allow sharing when the resource's semantics permit it. Many processes can read an immutable file simultaneously. A print spooler lets applications enqueue output and release their involvement; a dedicated service controls the physical printer. This changes the resource design rather than making simultaneous physical printing possible. **Limitation:** writable shared state and many physical devices still require exclusive access. Mutual exclusion cannot be removed indiscriminately.

4.2 Remove hold and wait

One policy requires a process to acquire *all* resources before beginning. A second permits a request only while the process holds none: release current resources and request the enlarged set together. **Why it works:** no process holds resources while blocking for more. **Costs:** a resource may sit unused while its owner performs unrelated work; future requirements may be unknown; a process needing a large combination may starve. Releasing partially used resources may also complicate consistency.

4.3 Allow resource preemption

If a new request cannot be satisfied, reclaim resources already held by the requesting process and restart it later when the required combination is available. Another policy reclaims suitable resources from blocked owners. **Why it works:** ownership is no longer irrevocable. **Limitation:**

state must be saved and restored safely. Reclaiming memory with an appropriate backing copy can be possible; interrupting a half-printed page cannot simply be undone. Arbitrarily taking away a mutex can expose an inconsistent data structure.

4.4 Remove circular wait by ordering

Assign each resource type a unique rank, and require acquisitions in strictly increasing rank order. If a process holds R_i, it may next request only an R_j with $\text{rank}(R_j) > \text{rank}(R_i)$. **Proof:** a cycle would require ranks to increase on every step and finally return to the initial rank. That would imply $r_0 < r_1 < \cdots < r_k < r_0$, which is impossible. Both threads in the earlier example must therefore acquire A before B. **Costs:** ordering may conflict with the natural order of work, and all code that obtains the resources must obey the same rule.

Condition attacked	Policy	Main drawback
Mutual exclusion	Share resources where safe	Many resources are inherently exclusive
Hold and wait	Acquire all together; or release before requesting	Low utilisation, unknown needs, possible starvation
No preemption	Reclaim and restore suitable resources	Unsafe for many devices and partial operations
Circular wait	Global resource ordering	Constrains every acquisition path

5 Avoidance and safe states

Avoidance permits more flexible acquisition but evaluates a request before committing it. It requires information about possible future needs, usually maximum resource claims.

Safe state

A state is safe if there exists an ordering of *all* processes in which each process can obtain its remaining maximum requirement using the resources currently free plus those released by earlier processes, complete, and return its resources. That ordering is a **safe sequence**.

The sequence is a proof that completion is possible under the declared claims; it is not an instruction that the CPU must run processes serially in that order. A process may request less than its maximum and finish sooner.

Safe implies no deadlock under the algorithm's assumptions. **Unsafe** means no such guarantee exists; it does not mean every process is already blocked. An unsafe state can still finish if actual requests are smaller than the declared maxima. Every deadlocked state is unsafe in this model, but some unsafe states are not yet deadlocked.

5.1 Single-instance avoidance using claim edges

A dashed **claim edge** $P_i \dashrightarrow R_j$ records that P_i may request R_j later. A claim becomes a request, then an assignment when granted, and returns to a claim when released. Claims must be declared under the protocol before acquisition. Grant a request only if converting it into an assignment leaves the augmented graph, including claims, acyclic. Here a cycle indicates an *unsafe potential* allocation; this test must not be confused with detecting an actual deadlock in a graph of current requests and assignments.

Further reading: OSC 9e, Chapter 7, Deadlock Avoidance; publisher's Chapter 7 slides.

5.2 Learn safe and unsafe with one resource type

Worked example: A safe allocation can become unsafe before anyone blocks

There are 6 identical units. P_0 holds 2 and claims a maximum of 5; P_1 holds 2 and claims a maximum of 4. Available is 2; their remaining maximum needs are 3 and 2. The state is safe: lend 2 to P_1, let it complete, then its release makes 4 available for P_0's remaining need of 3. Now tentatively give one available unit to P_0. Available falls to 1; P_0 holds 3 and still may need 2; P_1 holds 2 and still may need 2. Neither maximum requirement can be met first. The state is unsafe, even if both are currently computing and have not issued their next request.
It is not necessarily deadlocked: P_1 might actually require only one more unit and finish, releasing its resources. Avoidance cannot rely on that favourable future because the declared maximum permits it to ask for two. It therefore defers the grant that would remove the safety guarantee.

5.3 Why claim edges differ from current requests

For single-instance avoidance, suppose P_0 owns A and may later ask for B. P_1 is about to receive B and may later ask for A. Granting B creates the potential cycle

$$A \rightarrow P_0 \dashrightarrow B \rightarrow P_1 \dashrightarrow A.$$

The dashed edges describe possible future use, so this is an unsafe potential allocation. If those dashed edges were actual outstanding requests, both tasks would already be blocked and the cycle would identify current deadlock. The graph must say which kind of edge is present.

Edge	Meaning	When it changes
Claim $P \dashrightarrow R$	A possible future requirement	Becomes a request when issued
Request $P \rightarrow R$	Waiting for an instance now	Becomes an assignment when granted
Assignment $R \rightarrow P$	An instance is currently held	Removed on release; a continuing claim may remain

5.4 Read the safety algorithm as a proof search

Work is a *simulated available vector*. Selecting an eligible process means "this process could obtain everything still permitted by its claim and finish". Finish means the process has been accounted for in that proof, not that the real process was terminated.

If several processes are eligible, more than one safe sequence may exist. Do not reject a correct sequence because it differs from an answer's chosen ordering. Conversely, writing one eligible process proves only that one completion is possible; safety requires a completion ordering for *all* admitted processes.

6 Banker's algorithms

Banker's algorithm checks safety when resource types have several interchangeable instances. Think of the OS keeping enough flexibility to honour every admitted process's declared claim eventually.

6.1 Data and assumptions

For n processes and m resource types:

Name	Size	Meaning
Available	m	Unallocated instances of each type
Allocation	$n \times m$	Resources currently owned by each process
Max	$n \times m$	Declared maximum simultaneous claim
Need	$n \times m$	Remaining possible requirement: $Max - Allocation$

All entries are nonnegative, $Allocation \leq Max$, and each claim must be feasible relative to the total resources. Processes declare maxima before admission, remain within their claims, and return resources in finite time once their needs are met. The simple model assumes a known resource pool and interchangeable instances within a type. Vector comparisons mean *every* component must pass. For example, $\langle 1, 2, 0 \rangle$ is not at most $\langle 2, 1, 4 \rangle$, even though its sum is smaller.

6.2 The safety algorithm

```
Work = Available
Finish[i] = false for every process i
sequence = empty
repeat:
    find an i with Finish[i] == false and Need[i] <= Work
    if no such i exists: stop
    Work = Work + Allocation[i]
    Finish[i] = true
    append i to sequence
if every Finish[i] is true: SAFE
else: UNSAFE
```

Why add only $Allocation_i$? If the remaining need is lent from $Work$, the process returns both that loan and its original allocation on completion. Thus

$$Work' = Work - Need_i + (Need_i + Allocation_i) = Work + Allocation_i.$$

The safety check simulates completion; it does not really terminate processes or mutate the live allocation. A straightforward implementation takes $O(n^2 m)$ time in the worst case.

6.3 A complete safety example

Worked example: four processes, three resource types

Totals for A, B, C are $\langle 6, 4, 5 \rangle$. The initial snapshot is:

Process	Allocation	Max	Need
P_0	$\langle 1, 0, 1 \rangle$	$\langle 4, 2, 3 \rangle$	$\langle 3, 2, 2 \rangle$
P_1	$\langle 1, 1, 0 \rangle$	$\langle 2, 2, 2 \rangle$	$\langle 1, 1, 2 \rangle$
P_2	$\langle 2, 1, 1 \rangle$	$\langle 5, 3, 3 \rangle$	$\langle 3, 2, 2 \rangle$
P_3	$\langle 0, 1, 2 \rangle$	$\langle 1, 2, 3 \rangle$	$\langle 1, 1, 1 \rangle$

The allocated column totals are $\langle 4, 3, 4 \rangle$, so $Available = \langle 2, 1, 1 \rangle$. Initially only P_3 satisfies $Need \leq Work$. After its completion, P_1 becomes eligible.

Completes	Work before	Need	Released	Work after
P_3	$\langle 2,1,1\rangle$	$\langle 1,1,1\rangle$	$\langle 0,1,2\rangle$	$\langle 2,2,3\rangle$
P_1	$\langle 2,2,3\rangle$	$\langle 1,1,2\rangle$	$\langle 1,1,0\rangle$	$\langle 3,3,3\rangle$
P_0	$\langle 3,3,3\rangle$	$\langle 3,2,2\rangle$	$\langle 1,0,1\rangle$	$\langle 4,3,4\rangle$
P_2	$\langle 4,3,4\rangle$	$\langle 3,2,2\rangle$	$\langle 2,1,1\rangle$	$\langle 6,4,5\rangle$

All four can complete: $\langle P_3, P_1, P_0, P_2\rangle$ **is a safe sequence**. The final simulated Work equals the total resource vector, a useful arithmetic check.

6.4 The resource-request algorithm

For a live request $Request_i$, use this sequence:

1. If $Request_i \not\leq Need_i$, the request violates the declared maximum. Reject it as an invalid claim.
2. If $Request_i \not\leq Available$, the resources are unavailable. Make the process wait.
3. Otherwise tentatively allocate:

$$\begin{aligned} Available' &= Available - Request_i, \\ Allocation'_i &= Allocation_i + Request_i, \\ Need'_i &= Need_i - Request_i. \end{aligned}$$

4. Run the safety algorithm on this tentative snapshot. If safe, commit the grant. If unsafe, restore the original snapshot and defer the request.

Worked example: available resources can still be unsafe to grant

Start from the original snapshot above. P_1 requests $\langle 1,0,1\rangle$. This is at most its Need $\langle 1,1,2\rangle$ and the Available vector $\langle 2,1,1\rangle$. Tentatively,

$$Available' = \langle 1,1,0\rangle, \quad Allocation'_1 = \langle 2,1,1\rangle, \quad Need'_1 = \langle 0,1,1\rangle.$$

No process is eligible: P_1 and P_3 each still need one C, and P_0, P_2 need two. The safety test cannot start. **Defer the request and undo the tentative allocation.** The result is unsafe; it is not proof that an actual deadlock has already occurred.
For comparison, P_3 requesting $\langle 1,0,0\rangle$ is safe: Available becomes $\langle 1,1,1\rangle$, $Need_3$ becomes $\langle 0,1,1\rangle$, and P_3 can finish. Releasing its new allocation $\langle 1,1,2\rangle$ leaves Work $\langle 2,2,3\rangle$; then P_1, P_0, P_2 complete.

Banker answer checklist

Show Need, Available, each Work update, a complete sequence, and the final decision. For a request problem, perform *all three* tentative updates before testing safety. Do not approve a request merely because it is available, add Max to Work, or use a partially completed sequence as evidence of safety.

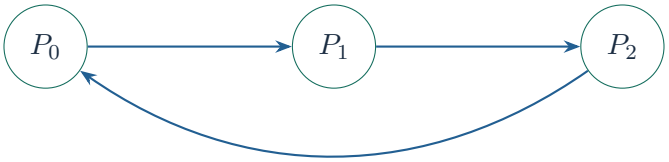
Practical limitations: maximum requirements may be unknown; resource availability may change; repeated checks cost time; conservative deferral can reduce utilisation. Banker is a clean model of avoidance, not a promise that every general-purpose OS uses it for all resources.

7 Deadlock detection

Detection allows allocation without maintaining a safe-state guarantee and periodically asks whether current waits can be resolved. It uses **present requests**, unlike avoidance's maximum future needs.

7.1 One instance: a wait-for graph

Remove resource vertices from the RAG. Draw $P_i \rightarrow P_j$ when P_i waits for the single instance owned by P_j. A directed cycle proves deadlock in this single-instance model.



Graph cycle detection can use depth-first search. The simple process-only cycle test is not sufficient for general multiple-instance allocation: a waiter may be satisfied by any of several owners.

7.2 Multiple instances: the matrix algorithm

Use Available, Allocation, and a matrix **Request** of currently outstanding requests. Request is neither Max nor Need.

```
Work = Available
Finish[i] = (Allocation[i] is the all-zero vector)
repeat:
    find i with Finish[i] == false and Request[i] <= Work
    if no such i exists: stop
    Work = Work + Allocation[i]
    Finish[i] = true
report every i with Finish[i] == false
```

Processes holding nothing cannot be holding up the resource cycle and begin marked finished for this test, although they may be blocked behind others. An eligible process is simulated to receive its current request, proceed, and release its allocation. The model assumes these present requests adequately describe the blocking situation; it is not an analysis of arbitrary later requests.

Worked example: a completion path that resolves the waits

There are two instances of A and one of B; Available is $\langle 0, 0 \rangle$.

Process	Allocation (A, B)	Current Request (A, B)
P_0	$\langle 1, 0 \rangle$	$\langle 0, 1 \rangle$
P_1	$\langle 0, 1 \rangle$	$\langle 1, 0 \rangle$
P_2	$\langle 1, 0 \rangle$	$\langle 0, 0 \rangle$

P_2 needs nothing more, so Work becomes $\langle 1, 0 \rangle$. Now P_1 can complete and release B, giving $\langle 1, 1 \rangle$. Finally P_0 completes, giving $\langle 2, 1 \rangle$. **No deadlock is detected**. The result depends on the actual Request matrix, not unknown future maximum claims.

In a second snapshot with only P_0 holding the sole A and P_1 holding the sole B, with the same cross-requests and no available resource, neither can pass the test. Both Finish entries remain false: $\{P_0, P_1\}$ **is deadlocked**.

7.3 When should detection run?

Checking after every failed request detects problems quickly but incurs overhead. Periodic checking reduces overhead but allows deadlocks and blocked queues to accumulate. A system can also trigger checks when progress or utilisation falls unexpectedly. Frequency depends on how often deadlocks occur and how expensive delayed recovery becomes.

Feature	Banker safety test	Detection test
Question	Can all declared maximum needs eventually be met?	Can current outstanding waits resolve?
Demand matrix	$Need = Max - Allocation$	Current Request
Initial Finish	False for all processes	True for zero-allocation processes
Failure means	Unsafe allocation	Deadlock identified by the model
Response	Refuse/defer tentative grant	Recover from existing deadlock

8 Recovery from deadlock

Detection identifies the problem; recovery must break it. The main choices are process termination and resource preemption. Both may discard completed work or require application-level repair.

8.1 Process termination

Abort all deadlocked processes. This breaks the cycle immediately but loses the most work. **Abort one at a time.** Select a victim, terminate it, release its resources, and detect again until the cycle is broken. Repeated detection costs time, but fewer processes may need to be lost.

Victim choice can consider priority, work already completed, remaining execution time, resources held and still requested, the number of other processes affected, and whether the process is interactive or a batch task. One small resource holder may be a better victim than a nearly complete expensive computation. Termination still needs correct cleanup; application data may need transactions or repair rather than mere memory reclamation.

8.2 Resource preemption and rollback

1. **Select a victim:** choose an owner and resource whose removal has an acceptable cost.
2. **Roll back:** restore the victim to a saved consistent checkpoint where it can resume safely after losing the resource. Sometimes a full restart is necessary.
3. **Prevent starvation:** avoid always selecting the same victim. Include previous rollback count or waiting time in the cost rule.

Preemption is useful only when the resource and affected state support recovery. It is not safe to seize every device or lock automatically. A checkpoint is a saved state; rollback returns execution to that state and loses work performed after it.

8.3 Detection and recovery form a complete strategy

Detection can find a bad snapshot without explaining which action is cheapest. Recovery must choose an intervention and then reassess the remaining dependencies. Removing one victim breaks cycles involving that victim, but another independent cycle may survive.

Worked example: Two independent deadlocked pairs

Suppose P_0, P_1 are waiting for each other's locks, while P_2, P_3 form a separate cycle. Terminating P_0 can release resources needed by P_1, but it does not free the locks in the second pair. The system must detect again or otherwise account for both cycles. "Terminate one process" is not a general guarantee that every deadlock in the system disappears.

A rollback policy needs a **consistent checkpoint**. If a process has made externally visible updates that cannot be undone, simply restoring its registers and memory may not restore the whole operation. Transactions, compensation or application recovery can be needed. This is why victim choice includes the cost and reversibility of lost work, not just process priority.

Ignoring general deadlock analysis at one level also does not mean all acquisition is unchecked. An application can impose lock ordering even if the OS has no global Banker test for its locks. Prevention, avoidance and detection/recovery are strategies that can coexist across different subsystems.

9 Choosing a deadlock strategy

Strategy	Decision	Information	Trade-off
Prevention	Restrict acquisition rules	Resource ordering or usage policy	Strong guarantee, constrained utilisation
Avoidance	Grant only if resulting state is safe	Declared maximum claims	Flexible rules, checking cost and conservative waiting
Detection and recovery	Allow; identify and break deadlock	Current ownership and waits	Better freedom, possible lost work
Ignore at a chosen level	No general handling for that resource class	No full global analysis	Low overhead, recovery delegated to users or applications

The last choice is often called the *ostrich approach*. It is a policy option, not a claim that an OS has no deadlock protections anywhere. Different layers can use different strategies: lock ordering inside one subsystem, transactions in a database, and restart for a failed application.

A complete 9-mark theory answer

For "explain deadlock handling", begin with the definition and four conditions, distinguish prevention from avoidance, and explain the requested algorithm or recovery mechanism. Use a labelled graph or a numerical trace where appropriate. Finish with specific limitations. For a narrowly worded question, spend most of the answer on the requested method.

II Module 4A: Memory management

10 Address spaces and address binding

An **address space** is the set of addresses a process can use to refer to its code and data. The OS should let each process address its own memory, prevent access to another process's private memory, and allow physical memory to be allocated efficiently.

Logical or virtual address: the address generated during program execution. **Physical address:** the address of the corresponding location in main memory. In the execution-time translation model used here, the memory-management unit (MMU) converts the virtual address into the physical address on each access. The process works with its virtual addresses rather than the current physical placement.

Two processes can use the same virtual address and reach different physical locations because they have different translation state. Conversely, shared mappings can let different virtual addresses refer to the same physical frame.

10.1 When binding occurs

Binding time	Mechanism	Effect of changing placement
Compile time	Generate absolute addresses when placement is known	Recompile if the assumed location changes
Load time	Generate relocatable code; resolve placement at loading	Reload/relocate for a different location
Execution time	Hardware translates addresses while running	Process can move if mappings are updated

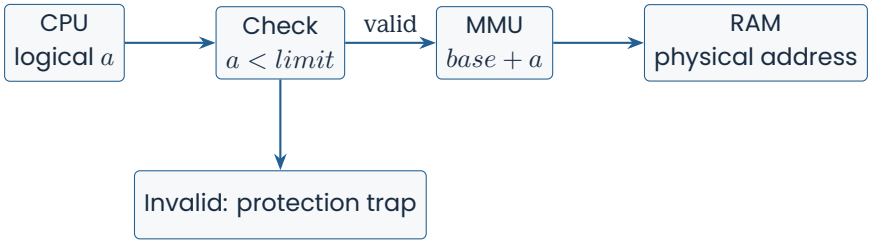
Do not confuse relocation with dynamically loading a routine when first called. Address binding answers *where an address refers*; loading answers *when code or data enters memory*.

10.2 Base and limit: the simplest translation

Let the base register contain the beginning of a process's physical region and the limit contain its length. For a byte address a,

$$0 \le a < limit \quad \Longrightarrow \quad physical = base + a.$$

An out-of-bounds reference traps instead of accessing memory. Only privileged OS code may modify the protection registers.



Worked example: relocation and the boundary

For $base = 12\,000$ and $limit = 2\,500$ bytes, logical address 875 becomes physical address $12\,875$. Address 2499 is the last valid byte and becomes $14\,499$. Address 2500 is invalid: the limit is a length, so the test is $a < limit$, not $a \leq limit$.

Further reading: OSC 9e, Chapter 8, Background; OSTEP Chapters 13 and 15.

10.3 Follow an address from source code to execution

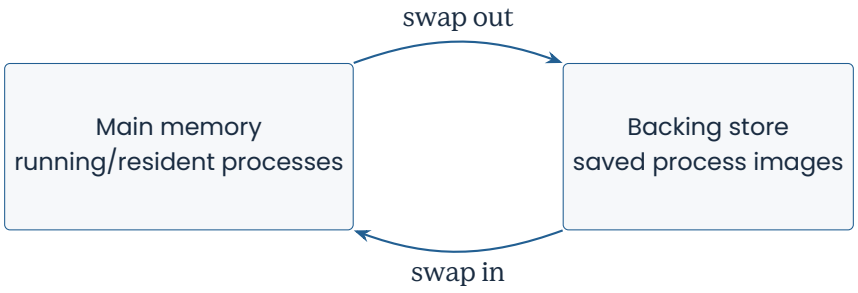
A source-level name such as `total` identifies an object. Translation/linking arrange its representation in a program image. Loading establishes the execution environment; relocation or runtime mappings determine where an access reaches physical memory. The exact toolchain varies, but symbolic names, virtual addresses and physical placement are different stages of description.

Execution-time translation lets process A and process B each use virtual address 1000 while reaching different physical memory. If A's page is moved, the mapping can change while A continues to use the same virtual address. Protection prevents A from freely replacing the mapping with B's private frame.

Dynamic loading brings a routine into memory when needed; **dynamic linking** resolves a relationship to library code at a later stage than static linking. Neither is another name for CPU scheduling or page replacement. These are supporting distinctions that explain why "the program is loaded" does not settle every address-management question.

11 Swapping

Swapping moves a process's memory image out of RAM to a backing store, freeing space, and later moves it back so execution can continue. In traditional whole-process swapping, the transferred unit is the process image. In demand paging, individual pages move; everyday terminology sometimes calls that swapping as well.



Relocation matters. With fixed physical bindings, the image generally needs its original placement. Execution-time translation allows a different placement on return if the translation state is updated.

I/O matters. A pending device transfer must not write into a region after that region is reassigned to another process. A system can keep the relevant pages pinned, defer removal until I/O completes, or transfer through kernel buffers.

Cost. Data transfer dominates whole-process swapping. If an image is S bytes and effective bandwidth is B bytes/s, one transfer takes approximately S/B, before extra latency. Swapping one image out and another in involves both transfers.

Worked example: transfer cost

An illustrative 32 MiB image at 128 MiB/s needs $32/128 = 0.25$ s in one direction. Exchanging two equal-sized images needs at least 0.50 s of transfer time, ignoring seek, queueing and other overhead. This explains why moving only necessary pages can be preferable.

Benefits: frees RAM and supports more admitted processes. **Costs:** expensive I/O, backing-store space, and added waiting. Swapping is not a CPU context switch: a context switch can occur while both processes' pages stay in RAM.

12 Contiguous allocation and partitions

In **contiguous allocation**, a process occupies one consecutive physical region. A base/limit pair can relocate and protect it. The OS maintains allocated regions and free regions called **holes**.

12.1 Fixed partitions

Divide available memory into partitions established in advance, either equal or unequal in size. One process occupies one partition. The number of resident processes cannot exceed the number of usable partitions, and a process must fit a partition. **Internal fragmentation** is reserved space *inside* an allocation that the process does not need. Putting a 70 KiB process in a 100 KiB partition wastes 30 KiB within that partition. Unequal partitions can reduce waste for varied process sizes but do not remove the fixed limits.

12.2 Variable partitions

Create a region sized for a process when it arrives. Split a suitable hole; when a process exits, return its region to the free list and merge adjacent holes. This avoids most fixed-partition slack, but leaves gaps of different sizes between allocations. **External fragmentation** means enough free memory exists in total but not as one sufficiently large contiguous hole. For example, separated holes of 60, 40 and 80 KiB total 180 KiB; none can hold a 100 KiB process. A realistic variable allocator can still have internal waste due to alignment or rounding.

12.3 First fit, best fit and worst fit

- **First fit:** choose the first hole in the searched list large enough. Here searches restart at the beginning for each request.
- **Best fit:** choose the smallest adequate hole. It leaves the smallest immediate remainder but may create many tiny holes.
- **Worst fit:** choose the largest hole, hoping to leave a useful large remainder. It can consume the large holes needed by later requests.
- **Next fit** is a supporting variation: like first fit, but resume the next search after the previous placement.

No strategy wins on every workload. The result depends on request order, future releases, and the free-list organisation.

Worked example: allocation with splitting

Initial holes in list order are $H_1 = 120$, $H_2 = 400$, $H_3 = 180$, $H_4 = 550$, $H_5 = 260$ KiB. Requests arrive as 210, 350, 170, 300 KiB. Holes split; there are no releases.

Request	First fit	Best fit	Worst fit
210	H_2; 190 remains	H_5; 50 remains	H_4; 340 remains
350	H_4; 200 remains	H_2; 50 remains	H_2; 50 remains
170	H_2; 20 remains	H_3; 10 remains	H_4; 170 remains
300	Fails	H_4; 250 remains	Fails

First fit ends with holes $120, 20, 180, 200, 260$: 780 KiB free, but no 300 KiB hole. Worst fit has $120, 50, 180, 170, 260$: again 780 KiB free, but no suitable hole. Best fit succeeds on this particular sequence. That is an example, not a universal ranking.

12.4 Coalescing and compaction

Coalescing combines *adjacent* free holes without moving allocated processes. **Compaction** moves allocated regions together so scattered holes become a large contiguous region. Compaction requires relocatable addresses, consumes copying time and must respect pinned memory/I/O. It treats external fragmentation; it does not eliminate unused space inside a fixed partition.

	Internal fragmentation	External fragmentation
Location of waste	Inside reserved blocks	Between allocated regions
Typical cause	Fixed units exceed actual needs	Variable contiguous allocations and releases
Example	Last partially used page	Many separated holes
Possible treatment	Smaller units or less rounding	Coalescing, compaction, noncontiguous placement

Further reading: OSC 9e, Chapter 8, Contiguous Memory Allocation; OSTEP Chapter 17.

12.5 Count allocated slack and free partitions separately

Worked example: A job may fail to fit even when much RAM is unused

Suppose 100 KiB is divided into a 20 KiB OS region and four fixed 20 KiB user partitions. Jobs of 12 and 18 KiB occupy two partitions, leaving 8 and 2 KiB of internal slack. Two entirely free partitions supply another 40 KiB.

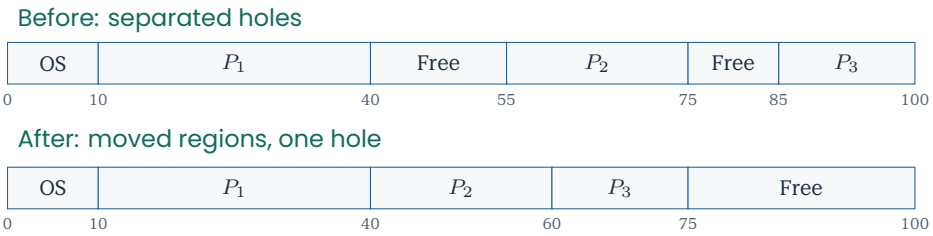
A 21 KiB job cannot occupy any single partition, although 50 KiB is unused overall. The 10 KiB *inside allocated partitions* is internal fragmentation. The 40 KiB in unallocated partitions is free capacity. Fixed boundaries prevent combining those partitions for this job. In the conventional textbook classification, internal waste is the characteristic fixed-partition problem; distinguish the separate maximum-partition-size constraint when a web description calls every aggregate failure “external fragmentation”.

Equal partitions simplify tracking but poorly match widely differing job sizes. Unequal partitions can accommodate a larger job and reduce slack for small jobs, yet assignment policy matters:

putting a small job into the only large partition can delay a large job. With fixed partitions the number of residents is also bounded by the number of usable partitions; variable partitioning instead creates regions according to actual requests.

12.6 See what compaction moves

Before compaction below, free holes are 15 and 10 KiB. A 20 KiB request fails despite 25 KiB total free. Coalescing cannot join the holes because P_2 lies between them. Compaction relocates allocated regions to create one 25 KiB hole.



Addresses are KiB boundaries. P_2 and P_3 change physical placement, so their relocation/mapping state must change correctly. Existing logical addresses must still refer to the intended objects. Compaction is copying and bookkeeping; coalescing only joins neighbouring free regions.

13 Segmentation

Segmentation divides a program's logical address space into meaningful variable-length units, such as code, global data, heap and stack. Each segment has a number and a length. In pure segmentation, each segment occupies a contiguous physical region; different segments need not be adjacent.

A logical address is (s, d): segment number and offset. The segment table stores a base address and a limit for each segment. Translation first checks that s identifies a valid segment and that the requested access is permitted, then checks

$$0 \le d < limit[s], \qquad physical = base[s] + d.$$

Protection can be attached to the logical unit: code read/execute, data read/write. Two processes can share a code segment while retaining private data segments.

Worked example: segment translation

Segment	Meaning	Base (bytes)	Limit (bytes)
0	Code	4000	1200
1	Data	9000	800
2	Stack	14000	600

$(1, 450)$ maps to $9000 + 450 = 9450$. $(0, 1199)$ maps to 5199. $(2, 600)$ is invalid because it equals the segment length. $(3, 0)$ is invalid because no segment 3 is defined.

Advantages: reflects program structure, supports segment-level sharing and protection, and lets segments grow separately if space permits. **Disadvantages:** variable physical regions suffer external fragmentation; growth and compaction are difficult; translation and segment-table management add overhead.

Segmentation is noncontiguous at the process level

Pure segmentation does not require the *entire process* to be contiguous, but each *individual segment* is contiguous. Segmentation with paging divides each segment into pages; mention it as a possible combination rather than confusing it with pure segmentation.

Further reading: OSC 9e, Chapter 8, Segmentation; OSTEP Chapter 16.

13.1 Segment-level protection and sharing

A code segment can be shared read/execute while each process retains a separate writable data segment. The same segment number in two processes need not identify the same physical region: their segment tables provide the interpretation. A program sees meaningful units rather than a single fixed-size partition for its whole image.

To answer a translation question, first verify the segment exists, then check the requested operation's permissions and the offset bound, then add base and offset. A plausible-looking numerical sum does not make an invalid address legal. An attempted write to read-only code can fail even with an in-range offset.

Pure segmentation reduces the requirement that *the whole process* occupy one region; it does not remove variable-size allocation problems for each segment. A long segment still needs an adequate hole and can cause external fragmentation. Paging a segment is one way to combine logical units with fixed-size physical allocation; it is a different model from pure segmentation.

14 Paging, page tables and the TLB

Paging divides virtual memory into fixed-size **pages** and physical memory into equally sized **frames**. A page can occupy any available frame; a process's frames can therefore be scattered in physical memory.

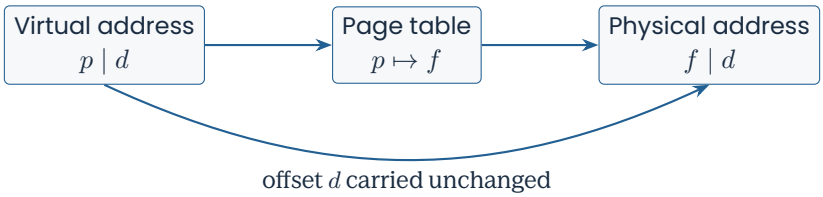
The **page table** records the page-to-frame mapping for a process. Paging removes the external-fragmentation requirement for ordinary single-frame allocation: every free frame can hold a page. It can still waste space inside the final partially filled page and consumes memory for page tables.

14.1 Translating an address

For a byte-addressed system with page size S,

$$p = \left\lfloor \frac{a}{S} \right\rfloor, \quad d = a \bmod S, \quad f = PageTable[p], \quad physical = fS + d.$$

The **offset is unchanged**. If $S = 2^k$, the low k address bits are the offset; the remaining high bits are the page number.



Worked example: page number, frame number and offset

Page size is 1 KiB = 1024 bytes. The table maps page 0 to frame 4, page 1 to frame 7 and page 2 to frame 5. For logical address 2365:

$$p = 2, \quad d = 2365 - 2(1024) = 317, \quad f = 5.$$

Hence the physical address is $5(1024) + 317 = \boxed{5437}$ bytes. Multiplying the *page* number by 1024 would reconstruct the virtual address, not the physical one.

14.2 Address bits, table size and fragmentation

In a byte-addressed virtual space with v address bits and 2^k bytes/page:

$$\text{offset bits} = k, \quad \text{page bits} = v - k, \quad \text{virtual pages} = 2^{v-k}.$$

If physical memory has 2^b bytes, the number of frames is 2^{b-k} and the frame number requires $b - k$ bits. A page-table entry (PTE) may need additional bits for protection and state; its size is not just the number of frame-number bits.

Worked example: linear page-table size

For a 32-bit virtual address, 4 KiB pages and 4-byte PTEs, $k = 12$, so there are 2^{20} possible virtual pages. A full single-level table occupies $2^{20} \times 4 = 2^{22}$ bytes = **4 MiB per process**. A 256 MiB physical memory has $2^{28-12} = 2^{16}$ frames, requiring 16 frame-number bits. The virtual page number has 20 bits; the two widths need not match.

For an allocation of L bytes, required pages are $\lceil L/S \rceil$, and last-page slack is $\lceil L/S \rceil S - L$. For $L = 10\,000$ bytes and $S = 4096$ bytes, three pages reserve 12,288 bytes, leaving 2288 bytes unused. The frequently quoted “half a page on average” assumes suitably distributed allocation sizes; it is not a fixed loss for every process.

14.3 What a PTE contains

A PTE commonly records a frame number, a present/resident bit, permission bits, a referenced/accessed bit, and a modified/dirty bit. A dirty bit means the in-memory data differs from its backing copy; it does not mean corrupt data.

Terminology differs between diagrams: some textbooks use valid/invalid for membership in the legal virtual address space; demand-paging diagrams sometimes also use it for residence. Conceptually distinguish **mapped and permitted** from **currently resident**. A legal page can be absent; an illegal address must not be satisfied by simply loading a page.

14.4 The translation lookaside buffer

If a one-level page table is in RAM, a simple uncached access needs one memory read of the PTE and another for the data. The **TLB** is a fast cache of recent page translations and permissions.

1. Search the TLB using the virtual page number and the relevant address-space identity.
2. On a hit, obtain the frame and check access rights.
3. On a miss, consult the page table; if the page is resident, install its translation and continue.
4. If the page is absent but the reference is legal, service a page fault.

A **TLB miss is not necessarily a page fault**. The page may already be in RAM. A TLB stores translations; a data cache stores data/instructions. At a process switch, translations must be separated using address-space identifiers or invalidated as required.

For TLB lookup time t, RAM access time m and hit ratio h, assuming a *serial* lookup, one-level RAM page table, no page faults and no other caches:

$$EAT = h(t+m) + (1-h)(t+2m) = t + (2-h)m.$$

Worked example: TLB effective access time

Take $t = 10$ ns, $m = 80$ ns and $h = 0.9$. A hit takes 90 ns; a miss takes 170 ns. Thus $EAT = 0.9(90) + 0.1(170) =$ **98 ns**. If a question explicitly ignores TLB lookup time, use $h\,m + (1-h)2m$ instead. State the assumed model.

14.5 Page-table organisation: supporting ideas

Multilevel tables divide the virtual page number into indices. An outer entry locates a lower-level table, whose entry locates the frame. Tables for unused address ranges need not be allocated. This saves space for sparse address spaces but can require extra reads on a TLB miss.

Hashed tables use a hash of the virtual page number (with process identity as needed) to find candidate mappings. **Inverted tables** hold roughly one entry per physical frame, describing which process/page occupies it; lookup needs a search/hash mechanism. These address page-table size at the cost of more complex lookup.

Further reading: OSC 9e, Chapter 8, Paging; OSTEP Chapters 18–20.

14.6 Why multilevel page tables save space

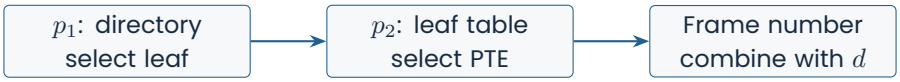
A full linear table reserves an entry for every possible virtual page, including huge unused address ranges. A multilevel arrangement allocates lower-level tables only where the process actually has mappings.

Worked example: A two-level 32-bit model

Use 4 KiB pages and 4-byte entries. Divide a virtual address as

$$\underbrace{p_1}_{10\text{ bits}} \mid \underbrace{p_2}_{10\text{ bits}} \mid \underbrace{d}_{12\text{ bits}}.$$

p_1 selects one of 1024 directory entries. That entry locates a leaf table; p_2 selects one of its 1024 PTEs. The PTE yields the frame; d is the byte offset. Each directory or leaf table occupies $1024 \times 4 = 4096$ bytes. One leaf maps $1024 \times 4096 = 4$ MiB of virtual addresses.
If mapped pages fall within just three distinct 4 MiB ranges, allocate one directory and three leaf tables: 4×4 KiB = **16 KiB**, rather than the full linear table's 4 MiB. This idealised count excludes other metadata and assumes those three leaf tables suffice. A densely mapped entire space may require every leaf, so the saving is workload-dependent.



Without caches, a two-level walk can read directory, leaf and data; a TLB hit bypasses the walk. Saving table memory is therefore traded against translation work on misses.

14.7 Hashed and inverted tables: understand the lookup key

A **hashed** arrangement hashes a virtual-page identity, checks matching entries in the bucket and retrieves a frame. Collisions require comparison of the actual identity; a hash result is not itself a physical address.

An **inverted** table describes physical frames rather than reserving one slot per virtual page. An entry records which address-space/page identity uses that frame. Because several processes can use the same virtual page number, process/address-space identity is essential to lookup. Its storage grows primarily with physical frame count, but finding a virtual page requires a search/hash arrangement and sharing needs appropriate extra handling. These are supporting implementations of the syllabus's paging abstraction.

14.8 Paging versus segmentation

Feature	Paging	Pure segmentation
Unit	Equal-size page	Variable-size logical segment
Address	Page number and offset	Segment number and offset
Table	Frame and state/permissions	Base, limit and permissions
Program structure	Pages need not match logical units	Segments reflect code/data/stack units
Physical placement	One frame per page	Contiguous region per segment
Main fragmentation	Internal	External
Protection; sharing	Per page	Per logical segment

15 Virtual memory and demand paging

Virtual memory separates the address space presented to a process from the amount and location of physical memory currently assigned to it. Only the actively needed portions must be resident. A process can therefore have a mapped virtual space larger than the RAM available to it.

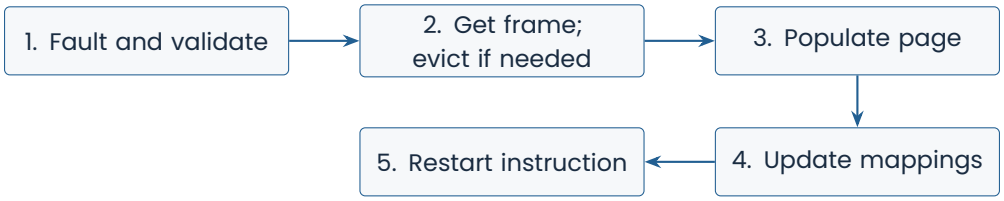
An absent page can be backed by an executable file, a mapped data file or swap space, or it may be generated as a zero-filled page on first use. Unmapped gaps do not need backing storage. Virtual memory is an address-management abstraction, not just "disk used as extra RAM".

Demand paging brings a legal absent page into a frame when it is referenced. It can reduce startup I/O and RAM usage because untouched pages need not be read. It also creates page-fault overhead and relies on hardware support for translation, protection and restarting interrupted instructions.

15.1 Servicing a page fault

1. A reference encounters a nonresident page; hardware transfers control to the OS.
2. The OS checks whether the address and requested operation are legal. An illegal reference is handled as an error, not a normal page-in.
3. Locate or create the page's contents and obtain a frame. If none is free, select a victim; preserve dirty data if necessary.

4. Arrange the read or other population operation. If waiting for I/O, block this process and let another runnable process use the CPU.
5. Update page/frame metadata and the relevant translations once contents are ready.
6. Restart the instruction so it can complete using the now-resident page.



The hardware must support a restart that preserves correct instruction semantics. An instruction that modifies several locations before faulting needs special care; blindly executing completed side effects twice would be wrong.

15.2 Paging and demand paging answer different questions

Paging defines fixed-size mappings between virtual pages and physical frames. Demand paging adds a *when-to-populate* rule: obtain a legal page's contents when access requires them. A system can use paging with all relevant pages resident; it can use demand paging to avoid loading untouched pages. Do not assume every page-table translation requires storage I/O.

In the ideal **pure demand** model, no process pages are populated before their first required references. Real systems can preload or read ahead some pages. A **minor** fault can be satisfied without a storage read, for example by a suitable zero-filled or already-available page; a storage-backed fault has a different cost. The exact categories and accounting are implementation-specific.

15.3 Build a reference string from byte addresses

With 4096-byte pages, byte references $0, 100, 4095, 4096, 8192, 4200$ generate page numbers $0, 0, 0, 1, 2, 1$. Divide each address by page size and discard the remainder only for the page identity. The remainder still identifies the byte inside the page.

Consecutive references to the same resident page do not cause repeated loads, but they are still memory references and hits. Do not drop them when calculating hit/fault ratios or updating policy state unless the question explicitly defines a compressed trace.

15.4 Why a dirty victim can make a fault slower

A free frame needs no eviction. A clean victim with a valid backing copy can often be discarded. A dirty victim whose changes must survive needs preservation before reuse. If an illustrative read costs 3 ms, a required dirty write 2 ms and other fault handling 0.1 ms, the serial model gives 3.1 ms for the clean case and 5.1 ms for the dirty case. Real paths can overlap work or use other backing arrangements; use the costs and model stated by the question.

15.5 Fault cost and effective access time

Let p be the probability of a fault per reference, m the normal access cost and F the *complete faulted-reference cost*, including service and restart. Then

$$EAT = (1 - p)m + pF = m + p(F - m).$$

If a question gives only an *extra* fault penalty D, use $EAT = m + pD$ instead. Do not count the normal access twice or omit it without stating the convention.

Worked example: rare faults are still expensive

Let $m = 100$ ns and $F = 8$ ms $= 8\,000\,000$ ns. For $p = 10^{-5}$,

$$EAT = (1 - 10^{-5})(100) + 10^{-5}(8\,000\,000) = 179.999 \text{ ns} \approx 180 \text{ ns}.$$

One fault per 100,000 references increases this idealised average by about 80%. To keep $EAT \leq 110$ ns,

$$p \leq \frac{110 - 100}{8\,000\,000 - 100} \approx 1.2500 \times 10^{-6}.$$

The 8 ms is an illustrative storage-backed fault cost, not a claim that every page fault performs disk I/O or has the same latency.

15.6 Locality, working sets and thrashing

Programs often reuse recently accessed data (**temporal locality**) and nearby locations (**spatial locality**). A **working set** is the set of pages used in a recent execution window. Its size changes as a program enters different phases.

Thrashing occurs when the resident allocation is inadequate for active working sets and frequent page turnover consumes much of the time. Evicting a page needed again soon produces another fault, displacing another useful page. CPU utilisation can fall while paging traffic rises. Adding more processes in response to the low CPU utilisation may worsen memory pressure.

Possible remedies are to assign enough frames to active working sets, reduce the number of competing resident processes, or use page-fault frequency to guide allocation. **Local replacement** chooses a victim from the faulting process's frames; **global replacement** may take a frame from another process. Global replacement can use RAM flexibly but lets one process disturb another's locality.

15.7 Frame allocation and the working-set argument

Equal allocation gives comparable processes similar frame counts. **Proportional allocation** divides a frame budget using an agreed measure such as process size. Allocation determines how many frames a process can use; replacement determines which resident page loses a frame. Global replacement also changes the distribution while choosing victims.

Worked example: Too many active working sets for the budget

Three processes have recent working sets of 4, 5 and 3 pages; the frame budget is 10. Their total target is 12, so all three sets cannot remain resident together. Suspending the third leaves a target of $4 + 5 = 9$, which fits the budget. This explains an admission remedy; a recent window is an estimate and does not guarantee every future reference will hit.

A common feedback mistake is: paging causes low CPU utilisation; the system admits more work to fill the apparently idle CPU; added work competes for frames and causes still more paging. The useful remedy is to examine memory pressure and active locality, not merely increase the number of runnable tasks.

A **page-fault-frequency** policy compares observed fault rates with thresholds. Too high a rate suggests more frames or reduced active load; a low rate may permit reclaiming frames. Thresholds and measurement windows matter. More physical RAM can help when capacity is the cause, but cannot replace correct allocation and workload control.

15.8 Copy-on-write: a useful supporting example

Processes can initially share a page with writes protected. When a private write is attempted, the OS creates a private copy and updates the writing process's mapping. This avoids copying pages that remain shared. The fault is used to implement a valid operation; not every protection-triggered fault means a programming error.

Further reading: OSC 9e, Chapter 9; OSTEP Chapter 21, Swapping: Mechanisms.

16 Page-replacement algorithms

When a legal absent page needs a frame and none is available under the allocation policy, **replacement** chooses a resident victim. A clean page with a usable backing copy can be discarded; dirty data that must survive needs preservation before its frame is reused. This avoids an unnecessary write for clean victims, but there is no universal percentage saving.

A **reference string** is a sequence of virtual page numbers, not byte addresses. A **hit** finds the page resident; a **fault** finds it absent. For the hand traces here, frames start empty, each first load counts as a fault, frame slots stay in a fixed order, and no prefetching occurs.

16.1 FIFO: replace the oldest load

Maintain the arrival order of resident pages. On a fault with full frames, evict the page that entered earliest. A hit does **not** change FIFO order. FIFO is simple but may evict a heavily used page simply because it has stayed resident a long time.

16.2 Optimal: replace the farthest future use

OPT replaces the resident page whose next reference is farthest in the future. A page never used again is a preferred victim. OPT gives the minimum possible number of faults for a given reference string and fixed frame count, but ordinary online execution cannot know all future references. Use it as a benchmark.

16.3 LRU: replace the longest unused page

Least recently used replaces the page whose most recent reference lies farthest in the past. **Every reference, including a hit, updates recency.** A timestamp or recency stack can represent the rule; exact tracking can be expensive, so practical schemes often approximate it.

The direction of the search

FIFO asks *when was it loaded?* LRU looks *backward*: when was it last used? OPT looks *forward*: when will it next be used? LRU is often effective because of locality, but it need not beat FIFO on every possible string.

16.4 Placement, replacement and allocation are different decisions

First/best/worst fit choose a free region in variable contiguous allocation. Page replacement selects a resident victim when a page needs a frame. Frame allocation distributes a budget across processes. They share the goal of managing memory, but their inputs and outputs are different; do not apply "best fit" to a FIFO page trace.

16.5 Explain the trade-off behind the replacement rule

FIFO has low bookkeeping cost but ignores whether its oldest page is still heavily used. LRU uses past recency because locality often predicts continued use, but exact recency tracking costs

time/state. OPT uses future knowledge and provides a lower-fault benchmark, rather than an ordinary implementable online guarantee. Clock samples use through reference bits to approximate useful history at lower cost.

LFU asks a different question: how often has the page been used? A page used intensely in an old phase can retain a large count while no longer being useful. Ageing or reset rules make the count respond to changing phases. MFU and MRU use different workload hypotheses and are not aliases for LRU. Unless a question specifies a policy's counting/tie rules, state yours before giving a trace.

The central check is whether the replacement rule preserves the exact intended metadata. FIFO hits leave load order unchanged; LRU hits update recency; Clock hits set the reference bit in the stated convention. A correct frame table with incorrect policy metadata can give the wrong victim several references later.

16.6 One verified reference string, three complete traces

Use three initially empty frames and the string

$$2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2, 1, 4, 1, 2.$$

Each row gives the state *after* that reference. "–" denotes an empty frame or no evicted page. Equal-distance OPT ties use the lower frame slot; another tie choice can change frame contents without changing the minimum fault total.

FIFO trace

Step	Reference	Frame 1	Frame 2	Frame 3	H/F	Evicted
1	2	2	–	–	F	–
2	3	2	3	–	F	–
3	2	2	3	–	H	–
4	1	2	3	1	F	–
5	5	5	3	1	F	2
6	2	5	2	1	F	3
7	4	5	2	4	F	1
8	5	5	2	4	H	–
9	3	3	2	4	F	5
10	2	3	2	4	H	–
11	5	3	5	4	F	2
12	2	3	5	2	F	4
13	1	1	5	2	F	3
14	4	1	4	2	F	5
15	1	1	4	2	H	–
16	2	1	4	2	H	–

FIFO: 11 faults, 5 hits. At step 3, referencing 2 is a hit and does not make it newly loaded. At step 5, 2 is still the oldest resident page and is evicted for 5. Step 6 therefore faults on 2.

LRU trace

Step	Reference	Frame 1	Frame 2	Frame 3	H/F	Evicted
1	2	2	–	–	F	–
2	3	2	3	–	F	–
3	2	2	3	–	H	–
4	1	2	3	1	F	–
5	5	2	5	1	F	3
6	2	2	5	1	H	–
7	4	2	5	4	F	1
8	5	2	5	4	H	–
9	3	3	5	4	F	2
10	2	3	5	2	F	4
11	5	3	5	2	H	–
12	2	3	5	2	H	–
13	1	1	5	2	F	3
14	4	1	4	2	F	5
15	1	1	4	2	H	–
16	2	1	4	2	H	–

LRU: 9 faults, 7 hits. At step 3, 2 becomes most recently used. At step 5, 3 is least recently used and is evicted. At step 6, 2 is a hit. This illustrates the difference between insertion order and access order.

OPT trace

Step	Reference	Frame 1	Frame 2	Frame 3	H/F	Evicted
1	2	2	–	–	F	–
2	3	2	3	–	F	–
3	2	2	3	–	H	–
4	1	2	3	1	F	–
5	5	2	3	5	F	1
6	2	2	3	5	H	–
7	4	4	3	5	F	2
8	5	4	3	5	H	–
9	3	4	3	5	H	–
10	2	4	2	5	F	3
11	5	4	2	5	H	–
12	2	4	2	5	H	–
13	1	4	2	1	F	5
14	4	4	2	1	H	–
15	1	4	2	1	H	–
16	2	4	2	1	H	–

OPT: 7 faults, 9 hits. At step 5, the resident pages are 2,3,1. Their next uses are steps 6,9,13 respectively, so OPT evicts 1. At step 7, among 2,3,5, their next uses are steps 10,9,8; OPT evicts 2.

Checking a hand trace

Count every F, including initial filling. Check that each referenced page is present after its row. On a hit, frame contents stay the same. FIFO age stays the same; LRU recency changes. There are 16 references, so faults plus hits must equal 16. The results here are FIFO 11, LRU 9, OPT 7.

16.7 Second chance and Clock

Second chance augments FIFO with a reference bit R. An access sets $R = 1$. During replacement, an old candidate with $R = 1$ has its bit cleared and receives another chance; a candidate with $R = 0$ can be replaced. A literal queue implementation moves spared pages to the back.

Clock implements the same principle with a circular list and a hand. On a fault, inspect the hand's frame: clear and skip $R = 1$ frames until reaching $R = 0$; replace that page, set the new page's $R = 1$, and advance the hand. On a hit, set $R = 1$ without moving the hand in the convention used here.

Worked example: Clock, including the reference bits

Use three empty frames. The hand begins at frame 1. Empty slots are filled in hand order; after a load the hand moves to the next frame. The string is $1, 2, 3, 1, 4, 2, 5, 1, 2, 3$.

Step	Ref.	Frames after access	R bits	Next hand	H/F
1	1	1,–,–	100	2	F
2	2	1,2,–	110	3	F
3	3	1,2,3	111	1	F
4	1	1,2,3	111	1	H
5	4	4,2,3	100	2	F
6	2	4,2,3	110	2	H
7	5	4,2,5	101	1	F
8	1	4,1,5	011	3	F
9	2	2,1,5	110	2	F
10	3	2,1,3	101	1	F

At step 5 the bits are initially 111. The hand clears all three and returns to frame 1, where it replaces 1 with 4. At step 6, 2 is a hit and its bit returns to 1. The hand remains at frame 2. The final total is **8 faults and 2 hits**.

Clock uses low-cost reference bits and approximates recency; it is not exact LRU. Specify initial hand position, bit-setting behaviour and whether the hand moves on a hit before comparing traces.

16.8 Other replacement ideas at revision depth

Enhanced second chance considers (R, M), where M is the dirty bit. The conceptual preference classes are $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$: favour pages not recently used and, other things equal, clean pages. Particular scans differ in how they clear R.

LFU evicts the least frequently used page; counts need aging or an old burst can protect a now-idle page. **MFU** evicts the most frequently used page, based on a different workload hypothesis. Frequency policies need a tie rule and a definition of when counts reset.

NRU periodically clears reference bits and chooses from the lowest available (R, M) class. **Aging** shifts sampled reference bits into small counters, giving recent samples more weight. A **working-set policy** tries to keep pages referenced in a recent window. These are supporting approaches; master FIFO, OPT, LRU and Clock first.

16.9 Belady's anomaly

Belady's anomaly is an increase in page faults when the same reference string is run with more frames. The conventional demonstration uses

$$1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.$$

FIFO has **9 faults with 3 frames**, but **10 faults with 4 frames**. Different capacity changes which pages are old at later references, so the larger FIFO cache need not contain the smaller cache's pages.

Algorithm	Victim criterion	Need future?	Belady anomaly?
FIFO	Oldest load	No	Possible
OPT	Farthest next use	Yes	No
LRU	Oldest last use	No	No
Clock	First eligible $R = 0$ at hand	No	Not excluded in general

For LRU, the resident set with k frames is contained in the set with $k + 1$ frames when running the same prefix; this is the **stack property**. LRU cannot exhibit the anomaly. OPT's minimum achievable fault count cannot increase with extra frames, since an extra frame can always be left unused. FIFO lacks the inclusion guarantee.

Further reading: OSC 9e, Chapter 9, Page Replacement; OSTEP Chapter 22, Swapping: Policies.

III Module 4B: File systems

17 The file concept

A **file** is a named collection of related information held on secondary storage. It gives applications a persistent, organised object instead of requiring them to manage physical sectors. A file system supplies naming, metadata, access operations, allocation and protection.

17.1 Attributes: what the OS records

Attribute	Meaning
Name	Human-readable name used in a directory
Identifier	Internal identity, such as an inode number
Type	Kind of object or expected format where supported
Location	Information needed to find its data blocks
Size	Current logical length, usually in bytes
Protection	Who may read, write, execute or perform other operations
Owner/group	Identities used for management and access checks
Timestamps	Creation, modification, access or metadata-change times as supported

The file's name may be stored in a directory while the remaining metadata is stored in a file control block (FCB) or inode. The exact metadata fields depend on the file system. An internal identifier is not necessarily unique across all mounted file systems without also identifying its volume.

17.2 Operations: what an application can request

- **Create:** allocate metadata and a directory entry; data space may initially be empty.
- **Open:** resolve the name, check permitted access, and create or obtain an open-file reference.
- **Read/write:** transfer data at the current offset or a specified offset, subject to access rights.
- **Seek/reposition:** change the current offset without necessarily transferring data.
- **Append:** write at the end; an append mode may enforce this placement.
- **Truncate:** reduce logical length, possibly to zero, while retaining the file object.
- **Close:** release the open-file reference and associated state.
- **Delete/unlink:** remove a name and eventually reclaim the object when no required references remain.
- **Get/set attributes:** inspect or modify metadata as permission rules allow.

Opening and deleting are more subtle than their names suggest

Opening usually establishes a handle and offset; it need not copy the entire file into RAM. In Unix-style semantics, unlinking the last name does not immediately destroy a file still held open: reclamation waits until the remaining references are released. Truncation retains the object/name while changing its length.

17.3 File types and internal structure

Types can include regular files, directories and special device files. Regular-file formats include source text, object code, executable programs, images and archives. A filename extension is a naming convention; it need not determine the contents or how the kernel treats the file.

Structure describes how contents are organised:

- **Byte stream:** an uninterpreted sequence of bytes. Applications define the meaning; this is the usual model for regular Unix files.
- **Record sequence:** fixed- or variable-length records, useful for structured data.
- **Indexed or tree structure:** records arranged for key-based retrieval.

These distinctions need not be exposed by the kernel. A database can implement records and indexes inside a byte-stream file. *Internal file structure*, *access method*, and *disk allocation method* describe different things.

Further reading: OSC 9e, Chapter 11, File Concept; OSTEP Chapter 39.

17.4 Separate a file's identity, type and contents

A name helps users find a file; an internal identifier helps the file system manage the object. A format describes how applications interpret bytes. These do not have a one-to-one relationship: two names can refer to one file object, and renaming `notes.txt` to `notes.jpg` does not convert the stored text into an image.

Example	Information represented	Useful distinction
Source/text file	Characters or program source	Usually interpreted by an editor/compiler
Object or executable file	Translated code and related metadata	Object code may need linking; an executable is prepared for the loader's format
Image, archive or data file	Encoded application information	The application defines the internal structure
Directory	Name-to-object associations	Organises lookup rather than behaving as an ordinary document
Special device file	An interface to a device where supported	Access can invoke device operations rather than ordinary stored-file reads

File-system formats such as FAT or an inode-based design describe *how storage and metadata are managed*. They are not the same classification as text versus image. Applications can store different content formats in the same file system.

17.5 A file-operation trace

Worked example: Offset, length and name are separate state

A 500-byte file is opened with an offset initially 0. Reading 80 bytes advances the offset to 80. Seeking to 200 changes it to 200 without reading the intervening bytes. Reading 20 bytes advances it to 220. Closing drops the handle but leaves the named file in place.
In a separate append operation, writing 10 bytes at the end grows the logical length from 500 to 510. Truncating to zero retains the file object/name but changes its length. Unlinking removes a name; remaining links or open references affect when its storage can be reclaimed under the chosen semantics.

18 File access methods

18.1 Sequential access

Read the next bytes or record, write the next data, and advance an offset. Rewind or skip operations may also be available. Sequential access fits logs, text streams and tape-like processing. Accessing a distant position solely by reading preceding records is inefficient.

18.2 Direct or random access

Select a byte offset or logical block/record number independently of the previous operation. For fixed-length records of size r, record i starts at byte offset ir when numbering begins at zero. This supports databases and large files requiring local updates.

Direct access describes the *interface*. Its efficiency depends on allocation: indexed or contiguous allocation can locate a target quickly; pure linked allocation must follow earlier links.

18.3 Indexed access

An auxiliary index maps a key or record identifier to a location. Search the index, then retrieve the selected record. This supports efficient lookup, often alongside sequential traversal. It adds storage and update overhead because the index must remain consistent with the data.

Worked example: direct record access

A file has 80-byte fixed-length records. With zero-based record numbers, record 125 starts at $125 \times 80 = 10\,000$ bytes. An application can seek to that offset and read 80 bytes. An indexed method could instead look up a student ID and obtain that record's location.

Method	Selection	Good use	Cost/limitation
Sequential	Next item at current position	Logs, streaming	Distant access may require scanning
Direct	Explicit offset/block	Random record operations	Requires efficient location mapping
Indexed	Key lookup through an index	Key-based data retrieval	Index storage and maintenance

18.4 Indexed sequential access combines lookup and traversal

An index can locate a starting record by key, after which records can be scanned sequentially. This suits a query such as "find the first account at this key, then read the following range". Updating records can require updating the index as well, so fast lookup trades against metadata maintenance.

Direct access uses a known position; indexed access first discovers a position from a key. They can be combined: after an index yields a byte offset, the program performs a direct read. For variable-length records, multiplying record number by a fixed length is invalid unless an appropriate position table or other mapping supplies the offset.

Three layers of file description

Structure: are the contents bytes, records or a tree?
Access method: does the caller read next, choose an offset or search an index?
Allocation: how are the resulting logical file blocks placed on storage?
A byte-stream interface can contain an application-built index; a direct read can still be expensive if locating its block requires traversing a linked allocation chain.

19 File protection

Protection decides which authenticated users or processes may perform which operations. Typical rights are read, write, execute, append, delete and list. Protection must be enforced on actual operations; hiding a filename alone does not protect the file.

19.1 Permission bits and access-control lists

Unix-style permission bits give read (r), write (w) and execute (x) rights to the owner, group and others. An **access-control list** (ACL) can attach rights to additional named users or groups, providing finer control than the three broad classes.

Worked example: reading a permission set

`rw-r-----` means: owner may read and write; group may read; others receive none of these permissions. Since $r = 4$, $w = 2$, $x = 1$, the octal representation is `640`.

Directory permissions have special meanings. **Read** lists entry names; **execute/search** permits traversal and lookup; **write**, normally together with search permission, permits changing directory entries. Deleting a Unix filename is primarily an operation on its *parent directory*; file write permission alone does not determine deletion rights. Additional rules such as the sticky bit can constrain deletion.

19.2 Protection, sharing and integrity

Shared access can use permissions to allow several readers while restricting writers. **Locks** or application protocols coordinate simultaneous changes, but locking is not a substitute for access control. Permissions say *who is allowed*; locking says *how permitted operations are coordinated*.

Apply least privilege: give only the rights necessary for the task. A shared account or weak authentication defeats a correct permission table. Encryption can protect the confidentiality of stored data, while backups help recovery; neither replaces operation-level access checks. This module's explicit protection scope is file protection rather than a full security-threat taxonomy.

19.3 Visualise rights with an access matrix

An **access matrix** describes subjects (rows), objects (columns) and allowed operations (cells). The following is an illustrative policy, not an operating-system default.

Subject	Notes	Shared report	Program
Student	read, write	read	read, execute
Project group	none	read, write	execute
Other users	none	none	execute

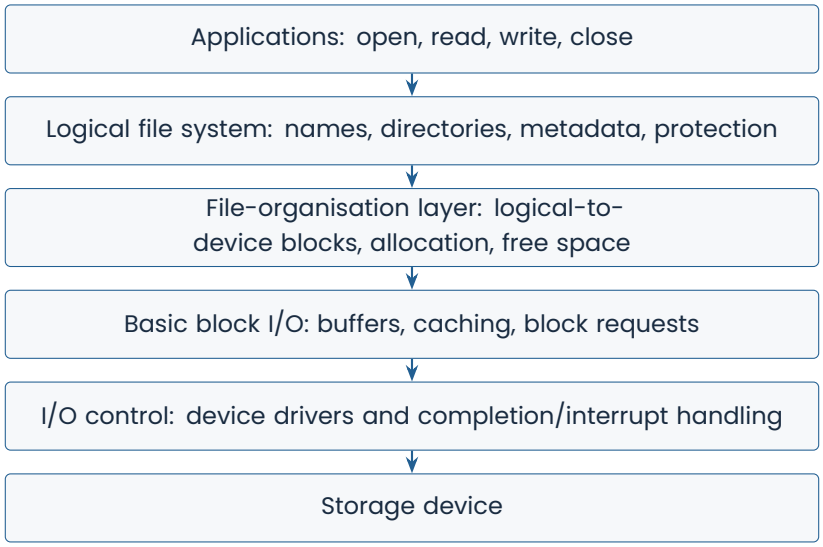
An ACL is an object-centred representation: the report's list says which subjects may read/write it. A **capability** is a protected or unforgeable authorisation identifying an object and permitted operations, held under the system's rules. A freely editable ordinary list is not itself a secure capability. This is supporting vocabulary for the basic question "who may do what to this file?"

Authentication establishes an identity; authorisation checks rights. A valid login does not imply permission to every file. Locks protect consistency among permitted users; they do not make an unauthorised access legitimate. A password attached to each file can be inconvenient to administer/share and does not substitute for a coherent access-control policy.

20 File-system implementation

The interface provides filenames and operations. Implementation translates these requests into metadata lookups, logical blocks and physical device I/O.

20.1 A layered view



This is a conceptual decomposition; actual implementations can combine layers. A virtual-file-system interface lets common operations reach different file-system implementations through a uniform abstraction.

20.2 On-disk and in-memory structures

On-disk structure	Purpose
Boot information, if present	Information/code used when booting from a volume
Volume control block or superblock	File-system size, block size and locations/state of key structures
Directory entries	Names and references to file objects
FCBs/inodes	Per-file metadata and data-location information
Free-space metadata	Which blocks or extents are available
Data blocks	File contents and supporting structures as allocated

RAM structures include a mount table, cached metadata and data, system-wide open-file state, and per-process descriptor tables. A descriptor locates an open-file object; the open object typically includes the current offset and access mode. Several references can share an open object and hence share an offset under the relevant interface semantics.

20.3 Following open, read and close

To open `/home/student/mos.pdf`, resolve each path component using directories, find the file's metadata, and check access. Create or reference an open object and return a descriptor. A subsequent read uses that descriptor and offset, maps the requested file blocks to device blocks, obtains data from cache or storage, and advances the offset as appropriate. Close drops the reference; the OS can release open state when its reference count permits.

Caching keeps recently used blocks in RAM, reducing repeated I/O. **Read-ahead** fetches likely future data for sequential access. Buffered writes can improve efficiency but require a persistence policy. A completed write to a cache does not necessarily mean the device has durably stored it.

20.4 Walk through a read as an implementation problem

1. On open, resolve path components from the root or current directory, using cached directory/metadata information where possible.
2. Locate the target file object and check the requested access under the file system's rules.
3. Establish an open-file object and a process-visible descriptor/handle referring to it.
4. On read, use that handle, access mode and requested range rather than resolving the filename from scratch for every byte.
5. Convert the file offset into logical block/offset information, then use the allocation mapping to locate storage blocks.
6. Obtain data from the block/page cache or arrange storage I/O. Waiting for I/O may block the caller.
7. Copy/deliver the requested data through the interface and update the offset as specified.

8. On close, release the reference; retain or reclaim shared open state according to remaining references.

Caching can remove physical reads from this path, but the logical distinctions still explain what information is needed. Successful access to cached data is not an exemption from the interface's protection rules.

20.5 Descriptors, open objects and file identity

Worked example: Sharing an offset is not the same as sharing a filename

Process A's descriptor 3 and B's descriptor 5 can refer to one shared open-file object with offset 8. If A reads 4 bytes through that object, its offset becomes 12; B's next ordinary read then uses that shared offset. In contrast, separate opens can create separate open-file objects with independent offsets while referring to the same underlying file.
A descriptor is a process-local handle. An open-file object holds active access state. An inode/FCB describes the persistent file. Multiple levels of references explain why copying/inheriting a descriptor can have different effects from opening the same pathname again.

20.6 Mounting connects a volume to a namespace

A file system must be recognised and made accessible through the OS naming arrangement. Mounting associates it with an appropriate location/identifier; path resolution can then cross into that file system. Formatting creates its structures, whereas mounting makes those structures accessible. Neither operation is simply an alias for opening one file.

20.7 Free-space management

The allocator needs to find unallocated blocks and return them when files shrink or disappear.

- **Bitmap/bit vector:** one bit per block. Consecutive set or clear runs, depending on convention, identify free extents. A bitmap is compact and supports contiguous-run searches.
- **Free list:** link free blocks or free-region records. Simple, but finding a long contiguous run can require traversal.
- **Grouping:** store several free-block addresses in one free block, with a link to the next group.
- **Counting:** store a start block and count for each free consecutive run; effective when space forms extents.

Worked example: bitmap storage

A 1 GiB volume with 4 KiB blocks has $2^{30}/2^{12} = 2^{18}$ blocks. One bit per block needs $2^{18}/8 = 2^{15}$ bytes = **32 KiB**, excluding metadata overhead. Here 1 means free; another text may use the opposite convention without changing the method.

Further reading: OSC 9e, Chapter 12; OSTEP Chapter 40, File System Implementation.

20.8 Practise a bitmap and a counted free-run representation

Worked example: One bit convention, two representations

Blocks are numbered 0–9. Let 1 mean free and use bitmap `0011100100`; free blocks are $2, 3, 4, 7$. A request for two consecutive blocks can take 2 and 3. The updated bitmap is `0000100100`, leaving 4 and 7 free.
Counting represents the original free runs as $(start, length) = (2, 3), (7, 1)$. After the same allocation, they become $(4, 1), (7, 1)$. It avoids one record per free block when many neighbouring blocks are free. A free list instead records/traverses available blocks or regions; grouping stores several free addresses together to reduce repeated traversal I/O.

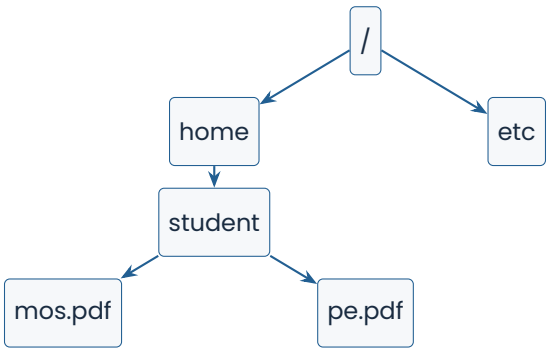
The allocator must keep free-space state consistent with file allocation. Marking an already-owned block free can give two files the same storage; failing to return a genuinely unused block leaks capacity. The updates need a coordinated protocol and, for persistent structures, an appropriate crash-recovery strategy. Full crash-consistency algorithms are beyond the listed syllabus, but the bookkeeping requirement explains why free-space metadata matters.

21 Directories and their implementation

A **directory** associates names with files or subdirectories. It supports lookup, entry creation, entry removal, renaming and listing. A path is resolved by repeatedly looking up its components.

21.1 Logical directory organisations

- **Single level:** one directory for every file. Simple, but users must avoid name collisions and cannot group files naturally.
- **Two levels:** a master directory selects a user's directory. Different users may reuse a filename; grouping within one user remains limited.
- **Tree:** subdirectories form a hierarchy. Absolute paths begin at the root; relative paths begin at the current working directory.
- **Acyclic graph:** sharing creates multiple paths to an object without cycles. Deletion requires tracking remaining references.
- **General graph:** cycles are possible. Traversal must avoid infinite looping, and simple reference counting can fail to reclaim cyclic unreachable structures.



If the working directory is `/home/student`, the relative name `mos.pdf` resolves to the absolute path `/home/student/mos.pdf`.

21.2 Implementing entries: linear list versus hash table

Linear list. Store entries containing a filename and file reference. An unsorted lookup examines entries until finding a match: $O(n)$ for n entries. Creation must also check duplicates. Deletion can mark an entry unused or reorganise the list. It is simple and compact but scales poorly for large directories.

Hash table. Hash the filename to locate a bucket and compare candidate entries. Expected lookup can be near $O(1)$ with a good hash function and controlled load, but collisions still require resolution. It consumes indexing space and may need resizing. A hash identifies candidates, not a guaranteed unique filename. Tree-based indexes are another implementation option.

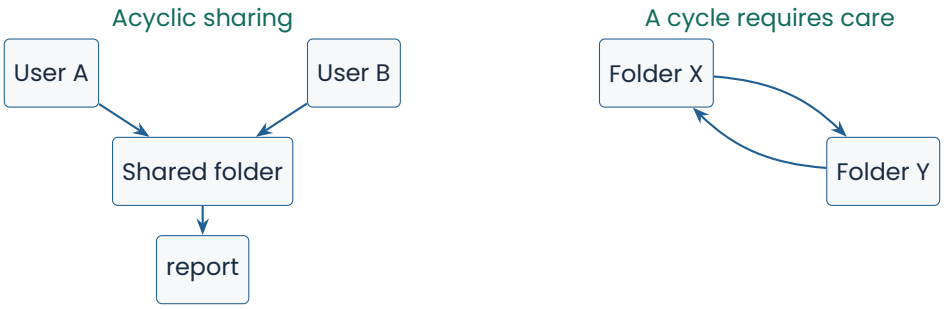
21.3 Hard links and symbolic links: supporting distinctions

A **hard link** is an additional directory entry for the same file object. Its inode link count tracks names; removing one name does not destroy the file while other names remain. A **symbolic link** is a separate object containing a pathname. Its target can disappear, leaving a dangling link, and path resolution must handle possible loops. These semantics illustrate why naming and object identity are different.

Further reading: OSC 9e, Chapters 11–12; OSTEP Chapter 39.

21.4 Evaluate directory designs by naming, grouping and sharing

Single-level naming is easy to implement but makes every filename compete in one namespace. Two levels separate users' names but provide little grouping within a user's own area. A tree adds nested groups and meaningful paths. Acyclic sharing allows multiple paths to a common object; it adds lifetime/reference management. General cycles need traversal rules that prevent endless revisits and a way to reclaim unreachable cycles.



These are abstract organisations. Actual file systems restrict which directory links are permitted; do not assume unrestricted directory hard links exist everywhere. Sharing one object saves duplication but also means one participant's changes can be visible through another path.

21.5 Directory organisation and lookup implementation are different

A tree describes relationships among directories. A linear list, hash table or search tree describes how entries within *one directory* are found. A hierarchical namespace can use hash-based entry lookup at every level; "tree directory" does not automatically imply linear or logarithmic lookup inside each directory.

Worked example: A hash collision still needs a name comparison

For illustration only, hash a name by the sum of its character codes modulo 4. `ab` and `ba` both sum to 195 and hash to 3. They remain different names. The bucket must distinguish them

using stored names, for example with a collision chain. Never treat the hash value alone as the file's identity.

22 File allocation methods

Allocation maps a file's logical blocks to blocks on the storage device. It is distinct from the access method presented to the application and from virtual-memory page allocation.

22.1 Contiguous allocation

Store a file in consecutive blocks. Metadata needs a starting block b and length n; logical block i maps to $b + i$, for $0 \leq i < n$. **Benefits:** fast sequential reads, simple mapping and efficient direct access. **Problems:** finding a sufficiently large run, external fragmentation, predicting file growth, and relocating a growing file when adjacent blocks are occupied. Reserving excess blocks can waste space.

Worked example: contiguous mapping

A file starts at device block 40 and occupies six blocks. Logical blocks 0–5 map to 40–45; logical block 4 is at block 44. Logical block 6 is outside this allocation.

An **extent** is a consecutive run described by a start and length. A file can use several extents, allowing growth without demanding one run for its entire final size. Extents retain locality but need additional mapping records.

22.2 Linked allocation

Each allocated block points to the next; the directory/FCB records the start, often also the last block. Data blocks may be anywhere on the device.



Logical block 2 in this chain is device block 1, reached by following links from the start. **Benefits:** easy growth and no external fragmentation at ordinary block granularity. **Problems:** slow direct access, space spent on pointers, and loss of the remaining chain if a link is damaged. There can still be unused bytes in the last block.

FAT variation. A file allocation table stores next-block links in a central table rather than in each data block. Keeping the table cached can speed traversal, but it consumes memory and does not turn the chain into a simple direct-index mapping.

22.3 Indexed allocation

An index block contains the addresses of the file's data blocks. Logical block i is obtained from index entry i rather than following all earlier data blocks. Entries may map, for example, $[9, 16, 1, 10]$; logical block 2 directly selects block 1 after locating the index. **Benefits:** direct and sequential access, flexible placement and growth. **Problems:** index-block space for small files, extra metadata I/O when uncached, and limits of a single index block for large files. Larger designs use linked index blocks, multiple levels, or a combination of direct and indirect pointers.

Worked example: one-level index capacity

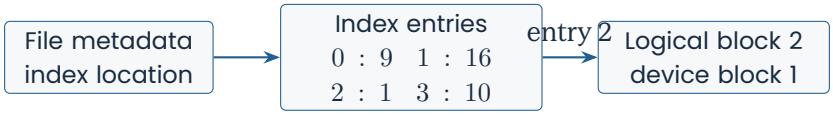
With a 4096-byte index block and 4-byte block addresses, there are $4096/4 = 1024$ entries if the whole block holds pointers. Each points to one 4096-byte data block. Maximum data addressed is $1024 \times 4096 = 4\,194\,304$ bytes = **4 MiB**. The index block itself is metadata, so it is not an extra data block in that result.

An illustrative inode with 12 direct pointers plus single and double indirect pointers can address

$$(12 + 1024 + 1024^2) \times 4096$$

bytes of data under those same pointer/block-size assumptions, excluding metadata blocks. This is a worked model, not a claim that every inode has those exact fields.

22.4 See the indexed mapping explicitly



The index stores locations, not the file's actual data. If not cached, obtaining an index can require metadata I/O before the requested data I/O. Linked allocation follows earlier next-block pointers; indexed allocation can select the appropriate mapping entry. Contiguous allocation computes it from start plus logical offset.

22.5 Pointer overhead can change the required block count

Worked example: Linked-block payload versus whole-block payload

A 1024-byte linked data block with a 4-byte next pointer carries 1020 data bytes. Storing 3070 bytes requires $\lceil 3070/1020 \rceil = 4$ such blocks. Pointer bytes total 16, and unused payload is $4(1020) - 3070 = 1010$ bytes.
A scheme using all 1024 bytes of each data block needs $\lceil 3070/1024 \rceil = 3$ data blocks, with 2 unused data bytes, though its mapping metadata is stored elsewhere. Compare both data-block usage and metadata overhead; "no external fragmentation" does not mean "no space cost".

22.6 Locate a block through direct and indirect pointers

For the illustrative inode with 12 direct pointers and 1024 pointers per index block:

Logical block numbers	Mapping path
0–11	Direct pointer in the inode
12–1035	Single-indirect block, entry $i - 12$
1036 onward, within double capacity	Double-indirect root, then a selected leaf index

For logical block 3000, subtract the earlier ranges: $3000 - 1036 = 1964$. Divide by 1024: root entry 1 identifies the leaf, and leaf entry 940 identifies the data block. A triple-indirect level would add another index step and capacity, but the actual layout depends on the chosen file system. Extra levels increase addressable size and potential uncached metadata reads.

22.7 Comparison you can use in an exam

Feature	Contiguous	Linked	Indexed
Location metadata	Start and length	Start plus next links	Index block(s)
Sequential access	Efficient	Natural but may scatter I/O	Supported; placement-dependent
Direct access	Simple $b + i$	Follow earlier links	Select index entry
Growth	Difficult if adjacent space used	Add another block	Add mapping, subject to index capacity
External fragmentation	Possible	No at block level	No at block level
Main overhead	Space reservation/relocation	Links and traversal	Index storage and lookup

A strong allocation-method answer

Draw the mapping, explain how logical block i is found, then discuss growth, access speed, fragmentation and metadata overhead. Do not say linked/indexed allocation has “no fragmentation” without qualification: the last block may still be partially unused, and scattered blocks can reduce I/O locality.

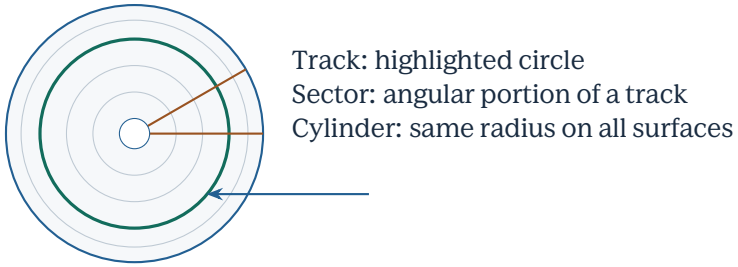
IV Module 4C: Storage management

23 Magnetic disks and disk structure

A magnetic hard disk stores data on rotating platters. A read/write head accesses each recording surface; an actuator positions the heads. These physical movements account for much of the latency of a small random request.

23.1 The geometry vocabulary

- **Platter:** a rotating disk with one or more recording surfaces.
- **Track:** a circular recording path on a surface.
- **Sector:** a subdivision of a track and a physical storage unit.
- **Cylinder:** the set of tracks at the same radius across surfaces.
- **Head:** the component reading/writing a surface.



The diagram is conceptual, not a particular manufacturer's layout. Modern interfaces expose **logical block addresses** (LBAs); drive firmware maps them to locations and handles details such as remapped bad sectors. A *file-system block* may comprise several device sectors. Track geometry is useful for the model, but an OS generally does not manage a literal platter map for each file.

23.2 Where an access spends time

$$T_{service} \approx T_{seek} + T_{rotation} + T_{transfer} + T_{controller}.$$

Seek time moves the head to the required track/cylinder. **Rotational latency** waits for the required sector to rotate under it. **Transfer time** moves the data. Response time also includes **queueing delay** before service begins.

For rotation speed R revolutions/minute,

$$T_{revolution} = \frac{60}{R}\,\text{s}, \qquad T_{average\ rotation} \approx \frac{30}{R}\,\text{s},$$

assuming random rotational arrival. For data size D and effective transfer rate B, $T_{transfer} \approx D/B$.

Worked example: a magnetic-disk service estimate

Take an illustrative 7200 rpm disk, 5 ms seek time, 100 MiB/s transfer rate and a 64 KiB request. One revolution takes 8.333 ms; average rotation waits 4.167 ms. Transfer takes

$64/(100 \times 1024)$ s $= 0.625$ ms. Ignoring controller time and queueing, total is approximately $5 + 4.167 + 0.625 =$ **9.792 ms**.

Large sequential transfers amortise positioning costs over many bytes. Scattered small transfers repeatedly pay them. Capacity and performance are different: a larger disk is not automatically faster for random I/O.

Further reading: OSC 9e, Chapter 10, Overview and Disk Structure; OSTEP Chapter 37.

23.3 Distinguish capacity, bandwidth and latency

Capacity measures how much data can be stored. Bandwidth measures transferred data per time. Latency measures how long one request waits to complete. A drive can transfer a large contiguous file efficiently yet have much higher per-request cost for scattered tiny reads because each pays positioning and queueing costs.

Worked example: A simplified constant-geometry capacity

Suppose a model has 1200 cylinders, 8 recording surfaces, 50 sectors per track and 512 bytes per sector. One cylinder includes one track per surface, so capacity is

$$1200 \times 8 \times 50 \times 512 = 245\,760\,000 \text{ bytes} = 234.375 \text{ MiB}.$$

This assumes the same sector count on every track and excludes metadata/spares. Real drives can use zoned recording and hide their physical layout behind LBA; do not infer modern capacity from a universal fixed track count.

A logical block number identifies a unit exposed by an interface. It need not reveal the head, recording surface or cylinder directly. The file system's block-to-device mapping and the drive's device-to-physical mapping are separate layers.

24 Solid-state disks

An SSD exposes block storage using semiconductor memory rather than rotating magnetic media. NAND-flash SSDs have no mechanical seek or rotational wait. Their performance still depends on the controller, flash operations, queueing and workload.

24.1 Flash pages, erase blocks and the FTL

NAND flash reads and programs at a **flash-page** granularity, but erases a larger **erase block** containing many flash pages. Updating programmed data generally requires arranging an erase before reprogramming; the device usually writes new data to another location instead of erasing the old block immediately.

The **flash translation layer** (FTL) maps host logical blocks to current physical flash locations. On an overwrite, it can write a new physical page, update the mapping and mark the old contents obsolete. This keeps the host's block interface stable.

- **Garbage collection** copies still-valid pages away from a chosen erase block, then erases the block for reuse.
- **Wear levelling** distributes erase/program activity because flash cells tolerate a limited number of such cycles.

- **Over-provisioning** reserves additional physical capacity for relocation, garbage collection and replacements.
- **TRIM/discard** tells the device which host logical blocks no longer contain needed data, helping it avoid preserving obsolete contents.
- **Write amplification** compares physical bytes written in flash with host bytes written. Internal relocation can make the former larger.

Worked example: write amplification

If the host writes 100 MiB while the controller writes 160 MiB including relocation, the write amplification for that interval is $160/100 = \mathbf{1.6}$. It depends on workload and device behaviour; it is not a universal fixed SSD constant.

Feature	Magnetic disk	NAND-flash SSD
Mechanism	Rotating media and moving heads	Semiconductor cells/controller
Positioning	Seek and rotation costs	No mechanical positioning
Updates	Sector writes under device rules	Programming plus block erase/relocation
Management	Geometry, defects, caching	FTL, wear levelling, garbage collection
Scheduling focus	Reduce movement and waiting	Queue management, parallelism, latency

Three different meanings of page/block

A virtual-memory page, a file-system block and a flash page belong to different layers. Their sizes need not be equal. An SSD still benefits from sensible I/O scheduling, but cylinder-seek optimisation is not its mechanical model.

Further reading: OSC 9e, Chapter 10, Solid-State Disks; OSTEP Chapter 44.

24.2 Follow an overwrite through the flash mapping

Suppose host logical block L maps to physical flash location A. An overwrite can program the new contents into available location B, change the FTL mapping to $L \mapsto B$ and mark A obsolete. Other still-valid pages in A's erase block prevent simply erasing that whole block immediately.

Garbage collection relocates those valid pages and eventually erases the block. These extra writes explain amplification. Wear levelling chooses placements/reuse to avoid exhausting one region prematurely. The host can keep asking for L without knowing which physical flash location currently holds it.

TRIM/discard reports that specified logical contents are no longer required; it can make internal reclamation more efficient. It does not itself promise that every old physical copy has immediately been securely erased. SSDs remove mechanical positioning costs, but still have finite bandwidth, erase/program costs, management overhead and workload-dependent latency.

25 Disk scheduling

When multiple magnetic-disk requests are pending, the OS or device scheduler chooses their service order. Objectives include shorter response time, higher throughput/bandwidth, fairness and reduced positioning work. For hand problems, total cylinder movement approximates seek work; it is *not* a complete service-time calculation.

25.1 The six standard algorithms

FCFS (first come, first served): service requests in arrival order. It is simple and respects arrival order, but distant alternating requests can cause large movements.

SSTF (shortest seek time first): choose the pending request closest to the current head position after *each* completion. It is locally greedy, not a proof of globally minimum travel. Under continuing arrivals near the head, distant requests can starve.

SCAN: move in a chosen direction, servicing requests encountered, reach the physical end and reverse. The elevator-style sweep reduces large arbitrary oscillations. Requests behind the head wait for a later sweep.

C-SCAN: service in one direction only. At the physical end, return to the opposite end without servicing requests on the return, then sweep again. It tends to make waiting more uniform across locations under an ongoing workload, at the cost of return travel.

LOOK: follow a SCAN-like sweep but reverse at the furthest pending request in that direction, avoiding travel to an unused physical end.

C-LOOK: service in one direction as in C-SCAN, but after the furthest pending request, jump to the extreme pending request at the other side and continue in the same service direction. The jump services no intermediate requests.

SCAN versus LOOK; circular versus bidirectional

SCAN visits the *physical endpoint*; LOOK visits the *furthest pending request*. C-SCAN and C-LOOK serve requests in *one direction*, whereas SCAN and LOOK serve on both legs. The circular return is still physical travel unless a question explicitly excludes it from the movement total.

25.2 A complete checked scheduling problem

Worked example: eight requests and an explicit convention

Cylinders are numbered 0–199. Head starts at 50, initially moving towards **larger** cylinder numbers. All requests are pending initially, in arrival order

$$40, 170, 90, 20, 150, 65, 110, 10.$$

No new arrivals occur. Count all movement, including endpoint travel before reversal and circular return travel; stop once the final request is serviced. For SSTF, break equal-distance ties by the smaller cylinder number (none affect this trace).

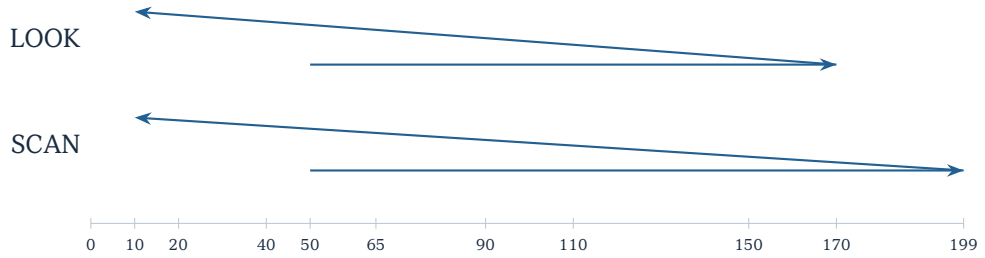
Policy	Complete head path (endpoints included)	Movement
FCFS	$50 \to 40 \to 170 \to 90 \to 20 \to 150 \to 65 \to 110 \to 10$	650
SSTF	$50 \to 40 \to 20 \to 10 \to 65 \to 90 \to 110 \to 150 \to 170$	200
SCAN	$50 \to 65 \to 90 \to 110 \to 150 \to 170 \to 199 \to 40 \to 20 \to 10$	338
LOOK	$50 \to 65 \to 90 \to 110 \to 150 \to 170 \to 40 \to 20 \to 10$	280
C-SCAN	$50 \to 65 \to 90 \to 110 \to 150 \to 170 \to 199 \to 0 \to 10 \to 20 \to 40$	388
C-LOOK	$50 \to 65 \to 90 \to 110 \to 150 \to 170 \to 10 \to 20 \to 40$	310

Total movement is $\sum_{i=1}^{k} |x_i - x_{i-1}|$, where x_0 is the initial head position and the path includes required endpoints/jumps.

- **FCFS:** $10 + 130 + 80 + 70 + 130 + 85 + 45 + 100 = 650$ cylinders.
- **SSTF:** after reaching 40, the nearest is 20, then 10, then 65; the full path totals 200.
- **SCAN:** $50 \to 199 \to 10$, so $(199 - 50) + (199 - 10) = 338$.
- **LOOK:** $50 \to 170 \to 10$, so $120 + 160 = 280$.
- **C-SCAN:** $50 \to 199$, return $199 \to 0$, then $0 \to 40$: $149 + 199 + 40 = 388$.
- **C-LOOK:** $50 \to 170$, jump $170 \to 10$, then $10 \to 40$: $120 + 160 + 30 = 310$.

If a problem excludes circular return distance, the C-SCAN result would instead be 189 and C-LOOK 150 for this snapshot. Those are *different accounting conventions*; use the one specified by the question and state it. For a finite queue we stop at the last service, so SCAN does not subsequently continue to cylinder 0 here.

25.3 A redrawable head-path diagram



The simplified strokes show sweep endpoints; all pending requests along a service sweep are handled. A graph drawn in an answer should label the cylinder range, start, initial direction, and request order.

25.4 Comparing algorithms fairly

FCFS is suitable when simplicity and arrival order dominate. SSTF reduces local travel but risks starvation. Sweep policies organise load and reduce arbitrary oscillation; LOOK avoids unnecessary endpoint travel, while circular variants target more uniform waiting. No ranking based on one numerical queue proves the best policy for every workload. Real scheduling also sees request sizes, reads/writes, caching, device reordering and deadlines.

Further reading: OSC 9e, Chapter 10, Disk Scheduling; UIC Mass-Storage notes.

25.5 Reconstruct each service policy from a decision rule

Policy	Next decision	What must be tracked
FCFS	Oldest pending request	Arrival queue
SSTF	Nearest pending cylinder	Current position; recomputed distances and tie rule
SCAN	Next request ahead; continue to physical boundary and reverse	Direction and physical range
LOOK	Next request ahead; reverse at furthest request	Direction and pending extremes
C-SCAN	Next request ahead; return from boundary to opposite boundary	One service direction; unserviced return
C-LOOK	Next request ahead; jump between pending extremes	One service direction; unserviced jump

SSTF must recompute distances after every service. Sorting once by distance from the *initial* head position implements a different rule. Circular return/jump is movement without service; count it if the movement convention includes physical travel.

25.6 Separate finite-snapshot answers from ongoing behaviour

For the supplied finite queue with no new arrivals, every algorithm eventually serves every request, assuming finite service. SSTF's starvation concern arises under continuing arrivals that repeatedly supply closer requests. SCAN's sweep and circular variants regulate opportunities over a changing workload; one snapshot's movement total does not establish the policy's long-run response distribution.

If head movement is converted to seek time using a given model, add rotational, transfer and controller costs only as specified. Real seek time need not be proportional to cylinders moved, and bytes transferred per second is not the same quantity as cylinders moved per request. Always attach units to the answer.

26 Formatting, partitions and boot information

Formatting has several distinct stages. Do not describe all of them as "deleting files".

26.1 Low-level or physical formatting

Prepare the device's physical sector organisation, including data areas and control/error-detection information under its technology. For modern hard disks this is normally performed by the manufacturer. A user-facing format command usually does not recreate the factory platter layout.

26.2 Partitioning

Divide a block device into logical ranges called partitions or volumes. A partition table records their boundaries and types. A partition can contain a file system, swap area or another format. Partitioning defines the ranges; it does not itself create all directory and allocation structures.

26.3 Logical formatting

Create the file-system structures in a chosen volume: superblock/volume information, free-space metadata, root directory, and metadata areas as the format requires. Logical formatting prepares the volume to hold named files.

An illustrative quick format primarily recreates metadata; a more extensive operation may also initialise or check data areas. Exact behaviour depends on the tool. Formatting is therefore not automatically a secure overwrite of all previous data.

26.4 Booting and bad-block handling

Firmware obtains initial boot code from an appropriate boot structure or loader. The loader locates the OS kernel, places it in memory and transfers control. A boot sector is the traditional simplified model; other boot schemes locate a loader in a dedicated boot partition/file. Not every partition must be bootable.

Devices use error-detection/correction information to identify and sometimes correct damaged data. A controller can replace defective sectors with spare sectors through remapping; software can also mark unusable blocks so the allocator avoids them. This changes the relationship between logical addresses and physical locations while preserving the host interface where possible.

A compact formatting answer

Physical formatting prepares sectors. Partitioning divides the device into volumes. Logical formatting creates a file system inside a volume. Boot structures locate initial executable code; bad-block management keeps defective locations from normal use. Explain the stages in this order.

26.5 Follow one drive through the preparation stages

A manufacturer prepares the physical medium's sector organisation. A partitioning tool defines two logical ranges. Logical formatting creates a file system in each chosen range, with the metadata that particular format requires. Mounting then makes a recognised file system accessible in the OS namespace. Boot configuration determines where executable startup code is found; a data-only volume need not be a boot volume.

A FAT-based format creates FAT structures; an inode-based format creates its own metadata. Do not describe every high-level format as necessarily initialising FAT. A format tool's checks, overwrites and recovery behaviour are tool/device-specific; formatting is not inherently a virus scan or a guaranteed secure erase.

Worked example: Volume-relative blocks versus device sectors

A partition begins at device LBA 2048. In this simplified model it uses 4096-byte file-system blocks on a device exposing 512-byte sectors, with block numbers relative to the partition start. Each block spans 8 sectors. Volume block 5 starts at

$$2048 + 5(8) = 2088$$

and occupies sectors 2088–2095. This arithmetic illustrates layer boundaries; locating a file's logical block still requires its allocation mapping before this volume-relative calculation.

V Revision and exam practice

27 Module 3: rapid revision

The essential chain

Deadlock needs **mutual exclusion, hold and wait, no preemption, circular wait**. Prevention disables a condition; avoidance preserves safety; detection uses current waits; recovery breaks the deadlock through termination or suitable preemption/rollback.

- A current RAG cycle proves deadlock when every resource type has one instance. With several instances, a cycle alone is insufficient.
- A safe sequence must include every process. Unsafe does not imply already deadlocked.
- In Banker, $Need = Max - Allocation$; start Work at Available and add *Allocation*, not Need or Max, when simulating completion.
- A request must pass the claim check, availability check and tentative-state safety check. Restore the state on an unsafe result.
- In detection, use *Request*, and initialise Finish to true for processes with no allocation.
- Recovery must consider victim cost, consistent rollback and repeated-victim starvation.

27.1 Three answer structures

Necessary conditions, 3 marks: define deadlock briefly; name and explain all four conditions in one sentence each. If the question asks only the conditions, do not spend the answer on unrelated algorithms.

Banker, 9 marks: define avoidance and safety; describe the four data structures; compute Need; show Work steps; give the safe/unsafe decision. If a request is asked, show tentative values and retest before deciding.

Prevention/recovery, 9 marks: define the problem, explain the requested policies with one concrete example, then state their limitations. Resource preemption is a recovery technique only when the affected state can be repaired.

28 Module 4: rapid revision

28.1 Memory

- A logical address is interpreted through a process's mapping. Physical placement can change without changing the logical address.
- Fixed partitions mainly cause internal fragmentation; variable contiguous regions mainly cause external fragmentation.
- Segmentation uses (s, d), base and limit. Paging uses (p, d), a frame map and fixed-size frames.
- Paging carries the offset unchanged: $physical = frame \times page\ size + offset$.

- A TLB miss may be resolved by a resident PTE. A legal absent page needs page-fault handling; an illegal access needs error handling.
- Demand paging loads on reference. Thrashing means page turnover overwhelms useful work.
- FIFO uses load order; LRU updates recency on every hit; OPT uses future references; Clock uses sampled reference bits.
- More frames can increase FIFO faults. OPT and LRU cannot exhibit Belady's anomaly.

28.2 Files and storage

- Attributes describe a file; operations manipulate it; access methods select positions; allocation methods locate blocks.
- Contiguous allocation gives simple direct lookup but difficult growth; linked allocation grows easily but direct access needs traversal; indexed allocation adds metadata but supports direct lookup.
- Directories map names to objects. A linear list is simple; hashing accelerates expected lookup but requires collision handling.
- Permission checks govern who may act; locking governs coordination of concurrent actions.
- Magnetic disks incur seek and rotational delay. SSDs use FTL mapping, garbage collection and wear levelling.
- SCAN reaches an end; LOOK stops at a request. Circular versions serve in one direction. State return-distance accounting.
- Physical formatting, partitioning and logical formatting are distinct stages.

29 Formula sheet

Task	Formula and condition
Banker remaining claim	$Need_{ij} = Max_{ij} - Allocation_{ij}$
Resource pool check	$Total_j = Available_j + \sum_i Allocation_{ij}$
Simulated completion	$Work' = Work + Allocation_i$, after $Need_i \leq Work$ (or $Request_i \leq Work$ for detection)
Contiguous relocation	$physical = base + a$, for $0 \leq a < limit$
Segment relocation	$physical = base[s] + d$, valid s and $0 \leq d < limit[s]$
Page and offset	$p = \lfloor a/S \rfloor$, $d = a \bmod S$
Paging translation	$physical = frame[p]\, S + d$
Address bits	For page size 2^k and v virtual-address bits: offset k, page number $v - k$
Number of frames	Physical-memory bytes / page-size bytes
Linear page table	Number of virtual pages $\times$ bytes per PTE
Allocation slack	$\lceil L/S \rceil S - L$ bytes
TLB EAT	$h(t + m) + (1 - h)(t + 2m)$; serial one-level lookup, no faults
Fault EAT	$(1 - p)m + pF$; F is complete faulted-reference cost
Fault rate bound	$p \leq (EAT_{max} - m)/(F - m)$
Bitmap size	Number of device blocks / 8 bytes, rounded up
Single index capacity	$\lfloor B/P \rfloor\, B$ data bytes, index block B bytes, pointer P bytes
Record offset	$i\, r$ for zero-based fixed-length record i of r bytes
Disk average rotation	$30/R$ seconds for R rpm, random angular arrival
Transfer time	Bytes transferred / bytes per second
Disk service time	Seek + rotation + transfer + controller overhead
Head movement	$\sum_{i=1}^{k} \lvert x_i - x_{i-1} \rvert$; include required endpoints and counted returns
Write amplification	Physical flash bytes written / host bytes written

30 Common mistakes and the correct statement

Mistake	Correct statement
“A graph cycle always proves deadlock.”	Only the single-instance current-RAG/WFG rule gives that conclusion without further instance analysis.
“Unsafe means deadlocked.”	Unsafe means no safe-sequence guarantee; actual needs may still be small enough to finish.
“Available means approve the request.”	Banker must test the tentative state’s safety too.
“Finish means terminate the real process.”	It records simulated completion during analysis.
“Detection uses maximum needs.”	Detection uses current outstanding requests.
“Preemption means CPU time-slicing.”	Deadlock’s condition refers to forcibly reclaiming held resources.
“Compaction fixes internal fragmentation.”	It joins external holes by moving allocated regions.
“Paging has no fragmentation.”	Ordinary frames avoid external fragmentation, but internal slack remains.
“Page and frame numbers are identical.”	The table maps one to the other; the offset stays the same.
“TLB miss implies disk I/O.”	The required page can already be resident.
“Virtual memory copies the whole address space to disk.”	Unmapped space requires no backing; pages can be file-backed or generated on demand.
“FIFO changes queue age on a hit.”	FIFO age is unchanged by a hit; LRU recency is updated.
“More frames always means fewer faults.”	FIFO can exhibit Belady’s anomaly.
“All page faults have the same disk latency.”	Faults may involve disk I/O, a zero fill, copy-on-write or an error, with different costs.
“Indexed access and indexed allocation are identical.”	One locates application records/keys; the other locates the file’s data blocks.
“Opening reads the entire file.”	It establishes an open reference; later accesses obtain contents.
“Deleting a name always destroys data immediately.”	Other names/open references may keep the object alive.
“An SSD has no scheduling cost.”	It lacks mechanical seek, but still has queues, controller work and flash operation costs.

Mistake	Correct statement
“C-SCAN’s return is zero distance.”	It is physical travel; excluding it is a specific accounting convention.
“Formatting is always secure erasure.”	Recreating metadata need not overwrite all old contents.

31 Part A practice: short answers with solutions

These are original 3-mark-style exercises. They cover the syllabus but are not predictions of the examination paper.

Q1. State the necessary conditions for deadlock.
Answer. Mutual exclusion: at least one resource is exclusive. Hold and wait: a process holds a resource while waiting for another. No preemption: ownership ends only by voluntary release. Circular wait: dependencies form a closed chain. All must coexist for reusable-resource deadlock.

Q2. Can a RAG contain a cycle without a deadlock? Explain.
Answer. Yes, with multiple instances. A process outside the cycle may release an instance needed by a process in the cycle, letting it finish and break the wait. With a single instance of every resource type, a directed cycle in the current RAG does imply deadlock.

Q3. Distinguish prevention from avoidance.
Answer. Prevention restricts acquisition so a necessary condition cannot hold, for example global lock ordering. Avoidance permits requests conditionally and grants only allocations that leave a safe state, using future-claim information such as Banker maxima.

Q4. What is a safe state? Is every unsafe state deadlocked?
Answer. A safe state has an ordering in which all declared remaining maximum needs can be met using current availability and earlier releases. Unsafe means this guarantee is absent, not that current waits are already deadlocked. Actual demands can be lower than maxima.

Q5. Why does the safety algorithm add Allocation instead of Max?
Answer. It lends the remaining Need from Work and receives it back along with the original Allocation at completion. Net Work changes by $-Need + Need + Allocation = Allocation$. Adding Max would overcount the loan.

Q6. Name the main deadlock recovery methods and a difficulty.
Answer. Terminate all deadlocked processes or selected victims, or preempt suitable resources and roll back affected processes. Recovery can lose work and must preserve consistency. Victim selection should avoid repeatedly penalising the same process.

Q7. Distinguish internal and external fragmentation.
Answer. Internal fragmentation is unused space within an allocation, such as slack in the final page. External fragmentation is scattered free space between allocations that cannot meet a large contiguous request. Compaction helps external fragmentation; reducing allocation granularity can reduce internal slack.

Q8. Distinguish paging and segmentation.
Answer. Paging uses fixed-size pages mapped to equal-size frames; translation uses a page table and preserves the offset. Segmentation uses logical variable-size units with base and limit entries. Paging mainly has internal slack; pure segmentation mainly suffers external fragmentation.

Q9. What does a TLB do? Does a miss imply a fault?
Answer. The TLB caches page translations and permissions, avoiding many page-table reads. A miss requires obtaining the mapping from the page table; the page may already be in RAM. A page fault is needed when the reference is legal but the required page is not resident.

Q10. Explain demand paging in three stages.
Answer. An absent-page reference traps to the OS, which validates the address. It obtains a frame, preserves a victim if required, and populates the page. It then updates mappings and restarts the instruction. Illegal references receive error handling rather than normal page-in.

Q11. Define Belady's anomaly and name a policy that shows it.
Answer. It is more faults after increasing frames for the same reference string. FIFO can show it: the conventional string in Section 16 produces 9 faults with three frames and 10 with four. LRU and OPT do not show it.

Q12. List important file attributes and operations.
Answer. Attributes include identity/name, type, size, location, owner, permissions and timestamps. Operations include create, open, read, write, seek, close, truncate and delete. Attributes describe state; operations manipulate or inspect the object.

Q13. Compare linear-list and hashed directories.
Answer. An unsorted list is simple but lookup is linear in the number of entries. A hash table narrows the search by filename hash and can provide near-constant expected lookup, but uses extra indexing space and must handle collisions and changes in table load.

Q14. Why is linked allocation poor for direct access?
Answer. To find logical block i, the implementation normally follows links through preceding blocks from the file's start. Contiguous allocation computes start+i, while indexed allocation selects the corresponding pointer. Linked allocation remains useful for easy growth and sequential traversal.

Q15. What do directory read, write and execute permissions mean?
Answer. Read lists names; execute/search permits traversing and resolving entries; write permits changes to entries, normally also requiring search permission. Deletion rights largely depend on the containing directory and additional rules, not only the target file's write bit.

Q16. Distinguish seek time and rotational latency.
Answer. Seek moves the head to the target track/cylinder. Rotation waits for the desired sector to reach the head. Both add positioning delay before data transfer. SSDs do not incur these mechanical delays.

Q17. Why does an SSD need garbage collection?
Answer. NAND updates commonly write new physical pages and leave old ones obsolete. Erasure works on larger blocks, so the controller moves still-valid pages elsewhere and erases the chosen block to reclaim space. This can add write amplification.

Q18. Distinguish SCAN and LOOK, and explain the initial direction.
Answer. SCAN sweeps to the physical end before reversal; LOOK reverses at the furthest pend-

ing request. The initial direction determines which side is served first and can change the numerical movement. State it explicitly before tracing.

Q19. What are the two main levels of formatting?

Answer. Physical formatting establishes device-sector organisation and associated control information, normally at the factory for modern HDDs. Logical formatting creates file-system metadata, free-space structures and the root directory inside a chosen volume. Partitioning separately defines volume boundaries.

32 Part B practice: longer answers and numericals

32.1 Problem 1: prevention and its proof

Question (9-mark style). Explain deadlock prevention by attacking the four necessary conditions. Give the proof behind resource ordering and two practical limitations.

Answer. Use the prevention table in Section 4. Sharing removes exclusivity where semantics allow; all-at-once acquisition removes hold and wait; suitable reclaim/restart removes no preemption; strict ordering removes circular wait. In a cycle ranks would need to strictly increase and return to the starting value, a contradiction. Limitations include the impossibility of sharing some devices, low utilisation from early reservation, unknown future needs, unsafe preemption and acquisition paths that must all obey the order. A good answer explains policies and drawbacks, not merely the condition names.

32.2 Problem 2: a fresh Banker snapshot

Question. One resource type has 10 instances. Allocations for P_0, P_1, P_2 are $2, 3, 1$, and maxima are $7, 5, 3$. Determine whether the state is safe. Then decide whether P_0 may receive three additional instances immediately.

Answer. Available is $10 - (2+3+1) = 4$ and Need is $[5, 2, 2]$. A safe trace is: P_1 needs 2 and releases 3, taking Work from 4 to 7; P_0 needs 5 and releases 2, taking Work to 9; P_2 needs 2 and releases 1, taking Work to 10. Thus $\langle P_1, P_0, P_2 \rangle$ is safe.

The request of 3 passes claim ($3 \leq 5$) and availability ($3 \leq 4$). Tentatively, Available is 1, Allocation is $[5, 3, 1]$ and Need is $[2, 2, 2]$. No process can finish with Work 1, so the tentative state is unsafe. **Defer and restore** the original state.

32.3 Problem 3: multiple-instance detection

Question. Available is $(0, 0)$. Allocations for P_0, P_1, P_2 are $(1, 0), (0, 1), (1, 0)$. Their current requests are $(0, 1), (1, 0), (0, 0)$. Run detection and explain how this differs from Banker.

Answer. All hold resources, so Finish starts false for all. P_2 is eligible, releasing $(1, 0)$; Work becomes $(1, 0)$. P_1 becomes eligible, releasing $(0, 1)$; Work becomes $(1, 1)$. P_0 completes, making Work $(2, 1)$. No false entries remain, so the model detects no deadlock. Detection uses current Request rather than maximum Need and need not know declared future maxima.

32.4 Problem 4: paging and table size

Question. A byte-addressed system has 24-bit virtual addresses, 4 KiB pages, 64 MiB physical memory and 4-byte PTEs. Find page/offset/frame bits and a full linear table's size. If virtual page 3 maps to frame 9, translate virtual address 12,345.

Answer. The offset has 12 bits; the virtual page number has $24 - 12 = 12$ bits. Physical memory is 2^{26} bytes, so it contains 2^{14} frames and requires 14 frame-number bits. A linear table has 2^{12} entries, requiring $2^{12} \times 4 = 16$ KiB.

For 12,345, page $p = 3$ and offset $d = 12\,345 - 3(4096) = 57$. Physical address is $9(4096) + 57 =$ **36,921** bytes.

32.5 Problem 5: TLB and fault performance

Question. A serial TLB lookup takes 5 ns; RAM takes 60 ns; the hit ratio is 0.95. Assume a one-level RAM page table and no faults. Then separately consider a system with normal access cost 100 ns and complete faulted-reference cost 5 ms: what fault probability keeps EAT at most 120 ns?

Answer. TLB hit cost is 65 ns and miss cost is 125 ns. $EAT = 0.95(65) + 0.05(125) =$ **68 ns**. For the second model, convert 5 ms to $5\,000\,000$ ns and solve

$$p \leq \frac{120 - 100}{5\,000\,000 - 100} \approx 4.00008 \times 10^{-6}.$$

These are separate models; do not combine them unless a question specifies how their costs interact.

32.6 Problem 6: page replacement by hand

Question. Two frames start empty. Trace FIFO, LRU and OPT for $1, 2, 1, 3, 2, 4, 1, 2$. Explain why a hit matters differently in FIFO and LRU.

Step	Ref.	FIFO frames	LRU frames	OPT frames
1	1	$(1, -)$ F	$(1, -)$ F	$(1, -)$ F
2	2	$(1, 2)$ F	$(1, 2)$ F	$(1, 2)$ F
3	1	$(1, 2)$ H	$(1, 2)$ H	$(1, 2)$ H
4	3	$(3, 2)$ F	$(1, 3)$ F	$(3, 2)$ F
5	2	$(3, 2)$ H	$(2, 3)$ F	$(3, 2)$ H
6	4	$(3, 4)$ F	$(2, 4)$ F	$(4, 2)$ F
7	1	$(1, 4)$ F	$(1, 4)$ F	$(1, 2)$ F
8	2	$(1, 2)$ F	$(1, 2)$ F	$(1, 2)$ H

Answer. FIFO has 6 faults, LRU 7 and OPT 5. LRU updates recency on the step-3 hit, so it evicts 2 at step 4; FIFO still treats 1 as the oldest load. This example also shows that LRU need not outperform FIFO on every string.

32.7 Problem 7: file allocation and metadata

Question. Compare contiguous, linked and indexed allocation for a file that grows unpredictably and requires random access. For 2 KiB index/data blocks and 4-byte addresses, calculate the data capacity of a one-level index. Explain directory hashing.

Answer. Contiguous allocation gives simple direct lookup but growth may require relocation. Linked allocation grows easily, but direct access follows preceding links. Indexed allocation supports random access and scattered growth within its mapping capacity, at the cost of index space and possible metadata I/O; it is a reasonable choice for the stated requirements.

An index holds $2048/4 = 512$ addresses. It maps $512 \times 2048 = 1\,048\,576$ bytes = **1 MiB** of data. Directory hashing maps a filename to a bucket of candidate entries; collisions require comparing actual names. It accelerates name lookup, not the logical-to-data-block mapping itself.

32.8 Problem 8: a fresh disk scheduling queue

Question. Cylinders range 0–99; head starts at 40 moving upward. Pending requests are $15, 75, 35, 90$. Find FCFS, SSTF, SCAN, LOOK, C-SCAN and C-LOOK movement. Count circular returns and stop after final service.

Algorithm	Path	Movement
FCFS	$40 \to 15 \to 75 \to 35 \to 90$	180
SSTF	$40 \to 35 \to 15 \to 75 \to 90$	100
SCAN	$40 \to 75 \to 90 \to 99 \to 35 \to 15$	143
LOOK	$40 \to 75 \to 90 \to 35 \to 15$	125
C-SCAN	$40 \to 75 \to 90 \to 99 \to 0 \to 15 \to 35$	193
C-LOOK	$40 \to 75 \to 90 \to 15 \to 35$	145

Answer. The endpoint and return conventions explain the differences. For example, SCAN travels $(99-40)+(99-15)=143$; C-SCAN travels $(99-40)+99+35=193$. Movement in cylinders is not milliseconds unless a seek-time conversion is supplied.

32.9 Problem 9: device mechanisms and formatting

Question. Explain how an HDD and NAND-flash SSD service data, and distinguish physical formatting, partitioning and logical formatting.

Answer. For an HDD, describe platters, tracks, sectors and cylinders; explain seek, rotation and transfer costs. For an SSD, describe flash pages and erase blocks, out-of-place updates, FTL mapping, garbage collection and wear levelling. Physical formatting prepares sector-level organisation, partitioning records volume ranges, and logical formatting creates file-system metadata inside a range. Use the diagrams in the storage chapter, then state that cylinder-sweep policies target HDD positioning rather than SSD mechanical delays.

33 A final self-check

Before the exam, confirm that you can do the following without looking at a worked answer:

1. Explain all four deadlock conditions and sketch a RAG with and without an actual deadlock.
2. Run a Banker safety test, evaluate a fresh request, and run detection using current Request.
3. Explain termination, preemption, checkpointing and recovery starvation.
4. Translate contiguous, segmented and paged addresses, and distinguish internal/external fragmentation.
5. Trace a fault from validation through restart, and calculate EAT with consistent units.
6. Trace FIFO, LRU, OPT and Clock; explain why extra FIFO frames can hurt.
7. Compare file access methods, protection rules, directories and allocation mechanisms.
8. Explain HDD/SSD internals and trace all six disk schedules with stated assumptions.
9. Distinguish the three stages preparing a volume for files.

Use the coverage map for completion and the full chapters when a revision bullet is not enough to reconstruct an explanation.

34 Sources and the best resources for further study

34.1 Use resources in this order

1. **Your department syllabus** controls what to prepare. The supplied PEEOT521 syllabus extract preserves PDF pages 171–174 of the existing full curriculum. Module boundaries in other KTU courses, including CST305 or EET433, are not interchangeable with PEEOT521.
2. **The prescribed textbook** supplies the terminology closest to this course: Silberschatz, Galvin and Gagne, *Operating System Concepts*, 9th edition, Wiley India, 2015. Use Chapters 7–12 by title as mapped below. Publisher materials were consulted directly for the algorithms and implementation models; owning the book is not assumed for using this guide.
3. **OSTEP** is the best free extended reading here for understanding the mechanisms through examples. Use only the mapped chapters when time is limited, rather than reading the entire book before the exam.
4. **University lecture notes** offer a compact second explanation of the prescribed textbook topics. They are useful when one mechanism remains unclear.
5. **Extra web exercises and lectures** help practise. Keep the syllabus and textbook terminology as the reference when a page uses a different convention.

34.2 Prescribed textbook: chapter map and official materials

OSC 9e	Chapter title	Use for
7	Deadlocks	Conditions, graphs, prevention, avoidance, detection, recovery
8	Main Memory	Address binding, swapping, contiguous allocation, segmentation, paging
9	Virtual Memory	Demand paging, replacement, working sets and thrashing
10	Mass-Storage Structure	Disks, SSDs, scheduling and formatting
11	File-System Interface	File concept, access, directories and protection
12	File-System Implementation	Structures, directory implementation, allocation and free space

The official 9th-edition slide index supplies chapter slides and a PDF collection. Its Chapter 7–12 slides were downloaded to temporary working storage and checked during preparation. Selected Chapter 14 slides on access matrices, ACLs and capabilities were also checked for the brief supporting file-protection vocabulary; the rest of that chapter is not required here. They are not republished in this source bundle. Regional editions can differ; identify a chapter by its title as well as its number.

34.3 Tanenbaum and Bos: an excellent reference

Modern Operating Systems, 4th edition, gives a complementary explanation: Chapter 6, Deadlocks; Chapter 3, Memory Management; Chapter 4, File Systems; Chapter 5, Input/Output, particularly disks. The chapter map and edition were checked against the publisher's 4th-edition contents. These are optional reading pointers; a full copy of that book was not locally available or fully consulted for this guide.

Edition discrepancy in the syllabus

The reference table prints Tanenbaum's book as "6th Edition, Pearson, Global Edition, 2015". The publisher identifies the 2015 fourth edition, and also lists a fifth edition released in 2022 with copyright 2023. Treat the printed edition/year combination as inconsistent; use the book title and verified chapter titles when locating a copy. This does not change the listed syllabus topics.

34.4 Free textbook: OSTEP, by topic

Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. The authors' official book page provides the free chapters. Read selectively:

Topic	Recommended chapter link and purpose
Deadlock intuition	32: Concurrency Bugs — lock-order problems and the four conditions
Address spaces and relocation	13: Address Spaces; 15: Address Translation — the mapping abstraction
Segmentation	16: Segmentation — base/bounds per logical region
Variable allocation	17: Free-Space Management — splitting, merging and fit policies
Paging and TLB	18: Paging; 19: TLBs — mappings and translation performance
Demand paging and replacement	21: Swapping Mechanisms; 22: Swapping Policies — fault handling and replacement trade-offs
Magnetic disks	37: Hard Disk Drives — physical timing and request order
File interface	39: Files and Directories — names, descriptors and links
File implementation	40: File System Implementation — metadata, data placement and operation paths
SSDs	44: Flash-based SSDs — out-of-place updates and flash management

These chapter links are directly to the authors' copies. Their explanations go deeper than the syllabus in places; the coverage map identifies the scope to prioritise.

34.5 University notes: a compact alternate explanation

John Bell's University of Illinois Chicago course notes explicitly reference the 9th-edition textbook:

- Deadlocks: graphs, safe states, Banker and detection/recovery.
- Main Memory and Virtual Memory: translation, partitioning, demand paging and replacement.
- File-System Interface and Implementation: attributes, access, directories and allocation.
- Mass-Storage Structure: disk scheduling and management.

34.6 Supplementary practice and video resources

GeeksforGeeks provides convenient short practice pages. Their topic relevance and links were checked; the primary textbook materials, university sources and independently computed traces supplied the correctness checks for this guide.

- Banker's Algorithm: additional matrix practice.
- Page Replacement Algorithms: extra reference strings and policy recaps.
- Disk Scheduling Algorithms: extra scheduling exercises; check direction and endpoint conventions.
- NPTEL: Operating System Fundamentals, Prof. Santanu Chattopadhyay, IIT Kharagpur: use the relevant deadlock, memory and file-system lectures for a slower explanation.
- NPTEL: Introduction to Operating Systems, Prof. Chester Rebeiro, IIT Madras: complementary systems explanations. Course identity was verified; complete lecture videos were not reviewed during preparation.

34.7 Preparation and verification record

The controlling syllabus, source index pages, university notes, the listed OSTEP chapters, and publisher chapter materials were checked during preparation on 7 October 2026. Diagrams and prose in this guide are newly written. The examples are original except the explicitly identified conventional FIFO anomaly string.

The accompanying `verify_examples.py` recomputes Banker, detection, fit policies, replacement traces, disk paths and the numerical practice answers. The two editions use the same source content; only the colour theme changes. Section links, table contents, page rendering and font embedding are checked after compilation. This is a study guide matched to the syllabus, with no claim of teacher-specific answer keys or unpublished exam predictions.

35 GeeksforGeeks topic map and expanded lessons

The relevant GfG articles were compared topic by topic on 7 October 2026. The lesson text is independently written: these links provide a second explanation and practice, while the syllabus and textbook materials control scope and correctness. The source bundle contains the detailed change record.

Read the guide first; use an article for a second explanation

The expanded edition adds short introductory examples before larger problems, execution traces, mapping diagrams and advantages/limitations. You can study the mechanisms directly from the guide without having to assemble the explanations from websites.

35.1 Module 3: topic links

Topic and guide section	GeeksforGeeks article
Deadlock and four conditions — 2	Read the matching article
Deadlock, starvation and livelock — 2	Read the matching article
Single-resource counting bound — 2.2	Read the matching article
Resource-allocation graphs — 3	Read the matching article
Handling strategies — 4	Read the matching article
Prevention — 4	Read the matching article
Avoidance and Banker — 6	Read the matching article
Detection and recovery — 7	Read the matching article

35.2 Module 4: topic links

Topic and guide section	GeeksforGeeks article
Address spaces — 10	Read the matching article
Address binding — 10	Read the matching article
Memory management overview — 10	Read the matching article
Swapping — 11	Read the matching article
Fixed partitions — 12	Read the matching article
Variable partitions — 12	Read the matching article
Internal and external fragmentation — 12	Read the matching article
First fit — 12	Read the matching article
Best fit — 12	Read the matching article
Worst fit — 12	Read the matching article
Segmentation — 13	Read the matching article
Paging — 14	Read the matching article
Page-table entries and structure — 14	Read the matching article
Virtual memory — 15	Read the matching article
Demand paging — 15	Read the matching article
Page replacement — 16	Read the matching article
Second chance / Clock — 16	Read the matching article
Thrashing — 15	Read the matching article

Topic and guide section	GeeksforGeeks article
Working set — 15	Read the matching article
File attributes, operations, types and structure — 17	Read the matching article
File access methods — 18	Read the matching article
File protection — 19	Read the matching article
File-system implementation — 20	Read the matching article
Unix/inode model — 20	Read the matching article
Free-space management — 20	Read the matching article
Directory structure — 21	Read the matching article
Paths and directory lookup — 21	Read the matching article
File allocation — 22	Read the matching article
HDD, SSD and disk structure — 23	Read the matching article
SSD mechanisms — 24	Read the matching article
Disk scheduling — 25	Read the matching article
SSTF — 25	Read the matching article
Disk formatting — 26	Read the matching article

The file-protection link is to the security discussion in a broader implementation article; it is not a verified standalone file-protection lesson. The guide supplies the rights, ACL and capability explanation from textbook material. The website also includes material for other courses. Supporting sections in this guide explain the named mechanisms; the GfG index is not a replacement syllabus.